FUNDAMENTOS DE COMPUTADORES · TEMAS 3 Y 4

Ensamblador MIPS 32 y jerarquía de memoria

Los dos temas explicados desde cero, en orden y con ejemplos

♥ ✿ ♥

Tema 3 (I) · Tema 3 (II) · Tema 3 (III) · Diapositivas del libro (Stallings)
Cuadro resumen MIPS-R2000 · Tema 4 (I) · Jerarquía de memoria · Tema 4 (II) · Memoria caché

He juntado todos los PDFs de los dos temas en uno solo: lo que se repetía aparece una única vez, en el sitio donde tiene sentido. Los errores de las diapositivas están corregidos (los tienes listados al final de cada tema).

Índice

TEMA 3 · PROGRAMACIÓN EN ENSAMBLADOR MIPS 32

1. **¿Qué es el ensamblador?**
   Datos e instrucciones · instrucción máquina · ISA · CISC y RISC · ciclo de ejecución · niveles de lenguaje
2. **El procesador MIPS 32 y tu primer programa**
   Arquitectura · registros · mapa de memoria · estructura · directivas · syscall · pseudoinstrucciones
3. **Instrucciones que trabajan con registros**
   Carga inmediata y movimiento · aritméticas · lógicas y máscaras · desplazamientos · comparaciones
4. **La memoria: bytes, palabras y vectores**
   lb, lbu, lw, sb, sw, la · extensión de signo · big/little endian · alineación · vectores y cadenas
5. **Control de flujo: if, else, while, switch**
   Saltos condicionales e incondicionales · slt · traducir C a ensamblador
6. **Formato de las instrucciones**
   Campos · tipos R, I y J · codificar en binario · código de operación · ejercicios de diseño · camino de datos
7. **Modos de direccionamiento**
   Implícito, inmediato, registro, directo, indirecto, relativo (base, índice, PC, pila) · b frente a j
8. **Subrutinas y pila**
   jal y jr · convenio de registros · llamadas anidadas · push y pop
9. **Ciclo de instrucción del MIPS**
   Las 4 fases, instrucción por instrucción
10. **Consejos, fallos típicos y ejercicios para practicar**
    Con soluciones
11. **Chuleta de instrucciones**
    Las que tu profe marcó ★ y el cuadro resumen MIPS-R2000
12. **Erratas corregidas del Tema 3**
    Lo que estaba mal en las diapositivas

TEMA 4 · JERARQUÍA DE MEMORIA

13. **Tipos de memoria y jerarquía**
    Registros, RAM y disco · características · unidades · contar accesos · la jerarquía y por qué funciona
14. **La memoria principal**
    ROM y RAM · lectura · DRAM y SRAM · refresco · FPM, EDO, SDRAM, DDR · tipos de ROM
15. **La memoria caché: funcionamiento y rendimiento**
    Proximidad · acierto y fallo · ejemplo con y sin caché · tiempo medio de acceso · L1, L2, L3
16. **Organización de la caché**
    Bloques y líneas · correspondencia directa, asociativa y por conjuntos · sustitución · escritura
17. **Ejercicios de examen de caché**
    Los 6 ejercicios largos de las diapositivas, resueltos
18. **Chuleta y erratas del Tema 4**
    Todas las fórmulas en una página · lo que estaba mal en las diapositivas

CÓMO LEER ESTA GUÍA

Definición lo que tienes que saber decir en el examen. **Analogía** una imagen para entenderlo. **Ejemplo** resuelto paso a paso. **Ejercicio** de las diapositivas, con su solución. **¡Ojo!** trampas típicas. **Truco** atajos. Registros: `$t0` y `$8` son el mismo; el libro usa números y clase, nombres (tabla en el apartado 2.2).

1 ¿Qué es el ensamblador?

1.1 Datos e instrucciones

Un computador recibe dos tipos de información y devuelve **resultados**:

Datos números, letras… + **Instrucciones** qué hacer con ellos → **Computador** → **Resultados**

Los **datos** se representan en binario (lo visteis en el tema anterior: enteros, complemento a 2, caracteres…). La pregunta de este tema es: *¿y las instrucciones?* La respuesta es que **las instrucciones también son números binarios** y se guardan en la memoria junto a los datos.

ANALOGÍA

Piensa en un cuaderno de cocina. En unas páginas está la **receta** (instrucciones) y en otras la **lista de ingredientes** (datos). Todo está escrito con el mismo bolígrafo y en el mismo cuaderno: la memoria. La CPU es la cocinera que lee la receta línea a línea.

1.2 Modelo de programación de un computador

Para programar un computador necesitas saber qué te ofrece. Eso es su **modelo de programación**, formado por:

- **Juego de instrucciones** (ISA, *Instruction Set Architecture*): las órdenes que entiende. Cada instrucción lleva un **código de operación** (qué hacer) y otros elementos: registros, direcciones de memoria o números.
- **Elementos de almacenamiento**: registros, memoria y registros de los controladores de E/S.
- **Modos de ejecución** (por ejemplo, modo usuario y modo núcleo del sistema operativo).

1.3 La instrucción máquina

DEFINICIÓN

Instrucción máquina: operación elemental que el procesador puede ejecutar directamente.

Ejemplo en MIPS: sumar un registro *s* con un valor inmediato *i* y guardar el resultado en el registro *t*. En binario es:

001000 código de operación (addi)	sssss registro s	ttttt registro t	iiii iiii iiii iiii valor inmediato

y en ensamblador se escribe `addi t, s, i`. Es decir: **código de operación + operandos**, y los operandos pueden ser **registros**, **direcciones de memoria** o un **valor inmediato** (un número metido dentro de la propia instrucción).

♥ Propiedades de las instrucciones máquina

- Hacen **una única tarea sencilla**.
- Trabajan con un **número fijo** de operandos.
- Llevan **toda la información necesaria** para ejecutarse (son autocontenidas).

♥ Qué información lleva una instrucción

1. **La operación** a realizar.
2. **Dónde están los operandos**: en registros, en memoria o en la propia instrucción (inmediato).
3. **Dónde dejar el resultado** (como un operando más).
4. **Cuál es la siguiente instrucción**:
   - *Implícitamente*: la que viene justo después (un programa es una secuencia consecutiva de instrucciones).
   - *Explícitamente*: en las instrucciones de salto, la dirección de destino va como operando.

1.4 El juego de instrucciones (ISA)

DEFINICIÓN

ISA: conjunto de instrucciones de un procesador. Es la frontera entre el hardware y el software: el software solo "ve" el procesador a través de ella. Ejemplos: **80x86** (Intel/AMD), **MIPS**, **ARM** (móviles), **Power**.

Un juego de instrucciones queda **definido** por: el conjunto de instrucciones, su formato, los registros, los modos de direccionamiento y los tipos de datos.

♥ Características que lo describen

Característica	Qué significa
Operandos	Pueden estar en registros, en memoria o en la propia instrucción.
Direccionamiento de memoria	Casi todos direccionan **por bytes** (cada byte tiene su dirección) y tienen instrucciones para leer datos de varios bytes a partir de una posición.
Modos de direccionamiento	Dicen **dónde y cómo** encontrar un operando (capítulo 7).
Tipo y tamaño de operandos	bytes (8 bits), enteros (16, 32, 64 bits), coma flotante (simple, doble precisión).
Operaciones	Aritméticas, lógicas, de transferencia, de control…
Control de flujo	Saltos incondicionales, saltos condicionales y llamadas a procedimientos.
Formato y codificación	Longitud **fija** (MIPS, ARM: siempre 32 bits) o **variable** (80x86: de 1 a 18 bytes).

♥ Tipos de instrucciones

Casi cualquier ISA tiene estas familias (las vamos a ir viendo todas): transferencia de datos aritméticas lógicas desplazamiento y rotación comparación control de flujo (saltos y llamadas) conversión entrada/salida llamadas al sistema.

♥ Clasificación: CISC frente a RISC

Un juego de instrucciones se puede clasificar por su **complejidad** (CISC o RISC) o por su **modo de ejecución** (de pila, de registros, registro-memoria, memoria-registro…).

	CISC (Complex Instruction Set Computer)	RISC (Reduced Instruction Set Computer)
Nº de instrucciones	Muchas	Pocas (juego reducido)
Cómo son	Complejidad variable; algunas ocupan **más de una palabra**	Simples y **ortogonales**; todas ocupan **una palabra**
Operandos	Pueden operar directamente sobre memoria	Operan sobre **registros** (a memoria solo se accede con carga/almacenamiento)
Unidad de control	Más compleja → mayor tiempo de ejecución	Más sencilla y rápida
Diseño	Irregular	Compacto; el silicio que sobra se usa para más registros y memoria caché
Ejemplo	80x86	**MIPS**, ARM

¿Por qué nació RISC? Se observó que alrededor del **20 % de las instrucciones ocupa el 80 % del tiempo** de ejecución, y que el 80 % restante casi no se usa. Es decir, el 80 % del silicio estaba infrautilizado, era complejo y caro. Idea RISC: quedarse con las instrucciones sencillas y hacerlas muy rápido.

TRUCO

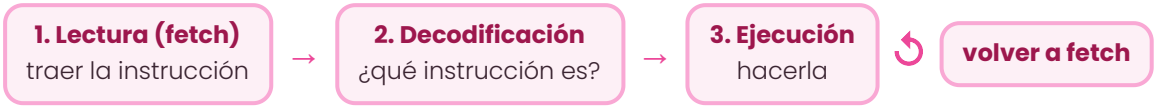
Ortogonal = todas las instrucciones usan los mismos modos de direccionamiento, sin excepciones raras. Como un juego de Lego en el que todas las piezas encajan con todas.

1.5 Programa y ciclo de ejecución

DEFINICIÓN

Programa: lista ordenada de instrucciones máquina que se ejecutan en secuencia (por defecto, una detrás de otra).

Cada instrucción pasa por estas fases, una y otra vez:



El **ciclo de fetch** en detalle, usando los registros internos de la CPU:

Paso	Qué pasa
`MAR ← PC`	El **PC** (contador de programa) guarda la dirección de la instrucción que toca. Se copia al **MAR** (registro de dirección de memoria).
Lectura	Se pide a la memoria el contenido de esa dirección.
`MBR ← Memoria`	Lo que devuelve la memoria llega al **MBR** (registro de datos de memoria).
`PC ← PC + 1`	El PC pasa a apuntar a la siguiente instrucción. *(En MIPS es `PC ← PC + 4`, porque cada instrucción ocupa 4 bytes.)*
`RI ← MBR`	La instrucción se copia al **RI** (registro de instrucción) para decodificarla.

ANALOGÍA

El PC es el **marcapáginas** de la receta: siempre señala el siguiente paso. Lees el paso (fetch), entiendes qué te pide (decodificas), lo haces (ejecutas) y mueves el marcapáginas.

1.6 Lenguaje ensamblador

DEFINICIÓN

Lenguaje ensamblador: lenguaje legible por un programador que es la representación **más directa del código máquina** de una arquitectura concreta.

- Usa **nemónicos** para las instrucciones: `add` (sumar), `lw` (cargar una palabra de memoria)…
- Usa **nombres simbólicos** para datos y referencias: `$t0` es un registro, `bucle` es una etiqueta…
- **Cada instrucción en ensamblador corresponde a una instrucción máquina**: `add $t1, $t2, $t3` es exactamente una instrucción de 32 bits. (La excepción son las *pseudoinstrucciones*, apartado 2.5.)

♥ Niveles de lenguaje

El mismo trozo de programa (intercambiar `v[k]` y `v[k+1]`) en los tres niveles:

Alto nivel (C)

```
temp   = v[k];
v[k]   = v[k+1];
v[k+1] = temp;
```

compilador →

Ensamblador (MIPS)

```
lw $t0, 0($2)
lw $t1, 4($2)
sw $t1, 0($2)
sw $t0, 4($2)
```

ensamblador →

Lenguaje máquina

```
0000 1001 1100 0110 …
1010 1111 0101 1000 …
1100 0110 1010 1111 …
0101 1000 0000 1001 …
```

Aquí `$2` guarda la dirección de `v[k]`; `v[k+1]` está 4 bytes después porque cada entero ocupa 4 bytes.

El **compilador** traduce de alto nivel a ensamblador y el **ensamblador** (un programa) traduce de ensamblador a binario.

EJEMPLO · COMPILAR Y DESENSAMBLAR

1) Escribes `hola.c` con `printf("Hola mundo…\n");`. 2) `gcc hola.c -o hola` genera el ejecutable (binario ilegible). 3) `objdump -d hola` lo **desensambla**: te enseña cada instrucción con su dirección, sus bytes en hexadecimal y su nemónico, por ejemplo `401051: 89 e5 mov %esp,%ebp` o `call 401130 <_printf>`.

Ese ejemplo está hecho en un PC Intel, así que el ensamblador que sale es de 80x86 (por eso las instrucciones ocupan 1, 2, 5… bytes: longitud variable). En la asignatura usamos MIPS.

1.7 ¿Para qué aprender ensamblador?

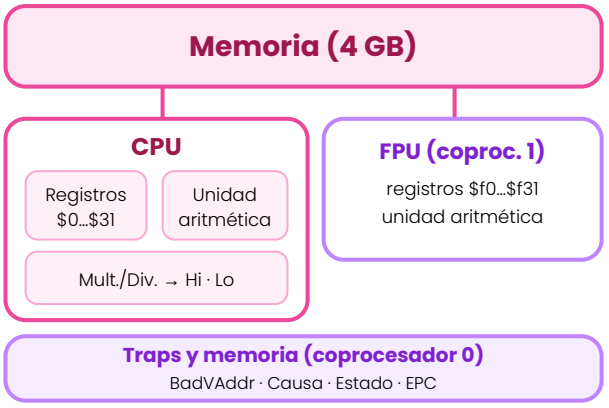
- Para **entender qué pasa de verdad** cuando el ordenador ejecuta una línea de C, C++ o Java.
- Para saber **cuánto tarda** una instrucción de alto nivel.
- Porque es útil en **compiladores, sistemas operativos, juegos y sistemas empotrados**.

Objetivos del tema: saber cómo se representan en ensamblador los tipos de datos (`int`, `char`…) y las estructuras de control (`if`, `while`…), y **poder escribir pequeños programas** en ensamblador.

2 El MIPS 32 y tu primer programa

2.1 Arquitectura del MIPS 32

- Procesador de **32 bits** de tipo **RISC**.
- **CPU**: 32 registros de enteros (`$0` … `$31`), unidad aritmética y una unidad de **multiplicación/división** con dos registros especiales, **Hi** y **Lo**, donde deja sus resultados.
- **Coprocesador 0**: gestiona excepciones, interrupciones y la memoria virtual (registros *BadVAddr*, *Causa*, *Estado* y *EPC*).
- **Coprocesador 1**: la **FPU** (unidad de coma flotante), con sus propios 32 registros.
- Direcciones de memoria de **32 bits** → 2^{32} bytes = **4 GB direccionables**.



2.2 El banco de registros

Hay **32 registros** de **4 bytes** cada uno (una **palabra** = 32 bits). Se escriben con **$** delante, por número (`$8`) o por nombre (`$t0`). Su uso viene dado por un **convenio**:

Nombre	Número	Uso
`$zero`	0	**Constante 0**: está cableado a cero y **no puede modificarse**
`$at`	1	Reservado para el ensamblador (lo usa al traducir pseudoinstrucciones)
`$v0, $v1`	2, 3	**Resultados** de una rutina (o de una expresión)
`$a0 … $a3`	4 … 7	**Argumentos** de entrada para rutinas
`$t0 … $t7`	8 … 15	**Temporales** (NO se conservan entre llamadas)
`$s0 … $s7`	16 … 23	**Valores guardados** (SÍ se conservan entre llamadas)
`$t8, $t9`	24, 25	Temporales (NO se conservan)
`$k0, $k1`	26, 27	Reservados para el sistema operativo
`$gp`	28	Puntero al área global (datos)
`$sp`	29	Puntero de **pila** (*stack pointer*)
`$fp`	30	Puntero al marco de pila (*frame pointer*)
`$ra`	31	**Dirección de retorno** de las rutinas (*return address*)

Agrupados: `reservados: $zero $at $k0 $k1` `argumentos: $a` `resultados: $v` `temporales: $t y $s` `punteros: $gp $sp $fp $ra`

ANALOGÍA

Los registros son la **encimera** de la cocina: poquito sitio (32 huecos) pero lo tienes a mano y es rapidísimo. La memoria es la **despensa**: enorme, pero tienes que ir a buscar las cosas (`lw`) y devolverlas (`sw`). La ALU solo trabaja con lo que hay en la encimera.

TRUCO PARA RECORDAR LOS NÚMEROS

temporales **t0** empieza en el 8 · **s**aved **s0** empieza en el 16 · los cuatro últimos son **gp sp fp ra** = 28, 29, 30, 31. Con eso deduces el resto.

2.3 Mapa de memoria de un proceso

Un programa en ejecución (proceso) divide su memoria en **segmentos** para organizarse:



- **Texto**: el **código** (las instrucciones). En el programa empieza tras `.text`.
- **Datos**: los **datos estáticos** (variables globales, cadenas…). Empieza tras `.data`.
- **Pila**: **variables locales** y el **contexto de las funciones** (direcciones de retorno, registros guardados…). Lo veremos en el capítulo 8.

2.4 Tu primer programa: "hola mundo"

```
.data                               # --- segmento de datos ---
    msg_hola: .asciiz "hola mundo\n"   # cadena acabada en 0

.text                               # --- segmento de código ---
    .globl main                     # main es visible desde fuera
main:
    # printf("hola mundo\n");
    li   $v0, 4                      # código 4 = imprimir cadena
    la   $a0, msg_hola               # $a0 = dirección de la cadena
    syscall                          # ¡pide al S.O. que lo haga!
```

Las piezas de un programa en ensamblador:

Pieza	Cómo se reconoce	Qué es
Directiva	empieza por `.`	Orden **para el ensamblador**, no para la CPU: no genera instrucciones. Ej.: `.data`, `.text`, `.asciiz`.
Etiqueta	acaba en `:`	Un nombre que **representa una dirección de memoria**. `main` = dirección donde empieza la función main; `msg_hola` = dirección donde empieza la cadena.
Instrucción	nemónico + operandos	Lo que ejecuta la CPU: `li $v0, 4`.
Comentario	desde `#` hasta el final de la línea	Para personas. ¡Úsalos mucho!

Nota: en las diapositivas a veces los operandos van separados por espacios (`li $v0 4`) y otras por comas (`li $v0, 4`). El simulador acepta las dos formas.

♥ Directivas más usadas

Directiva	Qué hace	Ejemplo
`.data` / `.text`	Empieza el segmento de datos / de código	
`.globl etiq`	Hace la etiqueta visible desde otros ficheros	`.globl main`
`.asciiz "…"`	Cadena terminada en un byte 0 (como en C)	`msg: .asciiz "hola"`
`.ascii "…"`	Cadena **sin** el 0 final	
`.byte`	Uno o varios bytes (8 bits)	`b1: .byte 0x00, 0x11`
`.half`	Medias palabras (16 bits)	`h: .half 7`
`.word`	Palabras (32 bits)	`v: .word 3, 7, 1`
`.space n`	Reserva *n* bytes (sin valor concreto)	`vec: .space 20`
`.align n`	Alinea el siguiente dato a una dirección múltiplo de 2^n	`.align 2` → múltiplo de 4

♥ Llamadas al sistema (syscall)

La CPU sola no sabe "imprimir" ni "leer del teclado": se lo pide al sistema operativo con `syscall`. Funciona así: **1)** pones en `$v0` el código del servicio, **2)** pones los argumentos en `$a0` (y `$a1`), **3)** `syscall`. Si el servicio devuelve algo, aparece en `$v0`.

Código en $v0	Servicio	Argumentos / resultado
1	imprimir entero	`$a0` = entero
4	imprimir cadena	`$a0` = dirección de la cadena (.asciiz)
5	leer entero	resultado en `$v0`
8	leer cadena	`$a0` = dirección del búfer, `$a1` = longitud
10	terminar el programa (exit)	—
11	imprimir carácter	`$a0` = carácter
12	leer carácter	resultado en `$v0`

EJEMPLO · LEER UN NÚMERO E IMPRIMIR SU DOBLE

```
.text
.globl main
main:  li   $v0, 5          # leer entero
       syscall               # ahora $v0 = número leído
       add  $a0, $v0, $v0    # $a0 = 2 * número
       li   $v0, 1           # imprimir entero
       syscall
       li   $v0, 10          # fin del programa
       syscall
```

Si tecleas 21, imprime 42.

2.5 Instrucciones y pseudoinstrucciones

DEFINICIÓN

Instrucción: se corresponde con **una única** instrucción máquina y ocupa 32 bits. Ej.: `addi $t1, $t0, 4`.

Pseudoinstrucción: se puede escribir en ensamblador, pero **no existe** como instrucción máquina. Al ensamblar se **sustituye** por una o varias instrucciones reales que hacen lo mismo.

Pseudoinstrucción	Se traduce a	Por qué
`li $v0, 4`	`ori $v0, $zero, 4`	0 OR 4 = 4
`move $t1, $t2`	`addu $t1, $zero, $t2` (o `add`)	0 + $t2 = $t2
`li $t1, 0x00800010`	`lui $t1, 0x0080` `ori $t1, $t1, 0x0010`	El número tiene 32 bits y **no cabe** en los 16 bits de inmediato: se carga en dos mitades

`addi $t0, $t0, 0xABABCDCD`	`lui $at, 0xABAB`
	`ori $at, $at, 0xCDCD`
	`add $t0, $t0, $at`
		Inmediato de 32 bits: se construye en `$at` (reservado para esto) y se suma

ANALOGÍA

Una pseudoinstrucción es como una **abreviatura** en tus apuntes: tú escribes "q" y lees "que". El ensamblador "expande" la abreviatura a palabras reales que la CPU sí entiende.

TRUCO · LUI + ORI

`lui` (*load upper immediate*) pone 16 bits en la **mitad alta** del registro y deja la baja a 0. Después `ori` rellena la **mitad baja**. Así: `lui $t1,0x0080` → `0x00800000`; `ori $t1,$t1,0x0010` → `0x00800010`. ✓

3 Instrucciones que trabajan con registros

En este capítulo todo pasa dentro de la CPU: los operandos están en registros o en la propia instrucción. La memoria llega en el capítulo 4.

CÓMO LEER UNA INSTRUCCIÓN MIPS

Casi siempre el **primer operando es el destino** y los demás son las fuentes: `add $t0, $t1, $t2` se lee "**$t0 ← $t1 + $t2**". Si la instrucción acaba en **i** (`addi`, `andi`, `ori`, `slti`…) el último operando es un **número inmediato** en vez de un registro.

3.1 Mover y cargar valores entre registros

Instrucción	Qué hace	Tipo
`move $a0, $t0`	Copia registro a registro: **$a0 ← $t0**	pseudo
`li $t0, 5`	Carga inmediata: **$t0 ← 5** (en binario 000…00101)	pseudo
`lui $s0, 12`	Pone 12 en los 16 bits **altos**; los 16 bajos a 0	I
`la $s0, VAR`	Carga la **dirección** de la etiqueta VAR (no su contenido)	pseudo
`mfhi $t0` / `mflo $t1`	Copia el registro especial **HI** / **LO** a un registro normal	R

3.2 Aritméticas

Hacen operaciones con enteros en la **ALU** (o con reales en la FPU).

Instrucción	Significado	Ejemplo con $t1 = 5, $t2 = 2
`add $t0, $t1, $t2`	$t0 ← $t1 + $t2	$t0 = 7
`addi $t0, $t1, 5`	$t0 ← $t1 + 5 (inmediato)	$t0 = 10
`sub $t0, $t1, $t2`	$t0 ← $t1 - $t2	$t0 = 3
`mul $t0, $t1, $t2`	$t0 ← $t1 · $t2 (pseudo)	$t0 = 10
`div $t0, $t1, $t2`	división **entera** (pseudo)	5 / 2 = **2**
`rem $t0, $t1, $t2`	**resto** de la división (pseudo)	5 % 2 = **1**

♥ Multiplicar y dividir "de verdad": HI y LO

Las instrucciones **reales** de la máquina tienen solo dos operandos y dejan el resultado en los registros especiales **HI** y **LO**:

- `mult $s0, $s1`: el producto de dos números de 32 bits puede ocupar **64 bits**. Los 32 bits altos van a **HI** y los 32 bajos a **LO**. (`multu`: igual pero sin signo.)
- `div $s0, $s1`: el **cociente** va a **LO** y el **resto** a **HI**. (`divu`: sin signo.)
- Después recuperas el resultado con `mflo` / `mfhi`.

EJEMPLO · 17 ENTRE 5

```
li   $s0, 17
li   $s1, 5
div  $s0, $s1      # LO = 3 (cociente), HI = 2 (resto)
mflo $t0           # $t0 = 3
mfhi $t1           # $t1 = 2
```

Las versiones de tres operandos (`mul`, `div $t0,$t1,$t2`, `rem`) son pseudoinstrucciones que el ensamblador traduce a esto.

TRUCO

Para no liarte: **LO**-**co**ciente (las dos con "o"), **HI** = el resto que "sobra arriba".

♥ Con signo y sin signo: el overflow

La aritmética puede ser en **complemento a 2** (con signo) o en **binario puro** (sin signo):

Instrucción	Qué hace
`add $t0,$t1,$t2`	Suma con signo (Ca2). Si hay **desbordamiento**, salta una excepción
`addi $t0,$t1,-5`	Suma inmediata con signo (el inmediato puede ser negativo)
`addu $t0,$t1,$t2`	Suma sin signo (binario puro): **nunca** avisa de overflow
`addiu $t0,$t1,2`	Suma inmediata sin signo
`sub` / `subu`	Lo mismo para la resta

SIN OVERFLOW

```
li   $t0, 0x7FFFFFFF
li   $t1, 5
addu $t0, $t0, $t1
```

Resultado `0x80000004`. Con `addu` no hay aviso aunque, leído en Ca2, el número haya "dado la vuelta".

CON OVERFLOW

```
li   $t0, 0x7FFFFFFF
li   $t1, 1
add  $t0, $t0, $t1
```

`0x7FFFFFFF` es el mayor positivo en Ca2 (2 147 483 647). Sumarle 1 con `add` provoca una **excepción de overflow**.

ANALOGÍA

Es como el cuentakilómetros de un coche con 6 cifras: después de 999 999 vuelve a 000 000. `addu` deja que dé la vuelta en silencio; `add` enciende una luz roja en el salpicadero.

EJEMPLO · I = A * (B + C)

```
int a = 5;
int b = 7;
int c = 8;
int i;
i = a * (b + c);
```

```
li  $t1, 5          # a
li  $t2, 7          # b
li  $t3, 8          # c
add $t4, $t2, $t3   # $t4 = b + c = 15
mul $t4, $t4, $t1   # $t4 = 15 * 5 = 75
```

Igual que en mates: primero el paréntesis y luego la multiplicación. El resultado (`i`) queda en `$t4`.

EJERCICIO · I = -(A * (B - 10) + C)

Con los mismos valores (a = 5, b = 7, c = 8). Primero sin `addi` y luego usando las instrucciones con inmediato.

SOLUCIÓN

Versión 1 (solo li y registros)

```
li  $t1, 5
li  $t2, 7
li  $t3, 8
li  $t0, 10
sub $t4, $t2, $t0   # 7-10 = -3
mul $t4, $t4, $t1   # -3*5 = -15
add $t4, $t4, $t3   # -15+8 = -7
li  $t0, -1
mul $t4, $t4, $t0   # -7*-1 = 7
```

Versión 2 (con inmediatos)

```
li   $t1, 5
li   $t2, 7
li   $t3, 8
addi $t4, $t2, -10  # -3
mul  $t4, $t4, $t1  # -15
add  $t4, $t4, $t3  # -7
mul  $t4, $t4, -1   # 7
```

Resultado: **i = 7**. Restar 10 es lo mismo que sumar -10, por eso basta un `addi`. (Otra forma de cambiar el signo: `sub $t4, $zero, $t4`, porque 0 - x = -x.)

EJEMPLOS DEL LIBRO

a = (b + c + d) - e: el ensamblador solo suma de dos en dos, así que se encadena:

```
add a, b, c     # a = b + c
add a, a, d     # a = b + c + d
sub a, a, e     # a = (b + c + d) - e
```

f = (g + h) - (i + j) con f, g, h, i, j en `$16, $17, $18, $19, $20` (= `$s0…$s4`). Se usan `$8` y `$9` (`$t0`, `$t1`) como temporales:

```
add $8,  $17, $18   # $t0 = g + h
add $9,  $19, $20   # $t1 = i + j
sub $16, $8,  $9    # f = (g + h) - (i + j)
```

3.3 Lógicas

Hacen operaciones booleanas **bit a bit**: el bit 0 con el bit 0, el bit 1 con el bit 1, etc.

Operación	Instrucción	En C	Ejemplo	Regla
AND	`and $t0,$t1,$t2`	`$t1 & $t2`	1100 AND 1010 = **1000**	1 solo si los dos son 1
OR	`or $t0,$t1,$t2`	`$t1 | $t2`	1100 OR 1010 = **1110**	1 si alguno es 1
XOR	`xor $t0,$t1,$t2`	`$t1 ^ $t2`	1100 XOR 1010 = **0110**	1 si son distintos
NOR	`nor $t0,$t1,$t2`	`~($t1\|$t2)`	1100 NOR 1010 = **0001**	el contrario del OR
NOT	`not $t0,$t1`	`~$t1`	NOT 10 = **01**	invierte cada bit (pseudo: `nor $t0,$t1,$zero`)

Con inmediato: `andi`, `ori`, `xori`. Ojo: el inmediato de 16 bits se extiende **con ceros** (no con signo). Ej.: `ori $t0, $t1, 80` → $t0 = $t1 | 80; `andi $t0,$a0,0xA1FF` → $t0 = $a0 & 0x0000A1FF.

EJERCICIO · ¿QUÉ VALE $T2?

```
li  $t0, 5
li  $t1, 8
and $t2, $t1, $t0
```

SOLUCIÓN

```
$t0 = 00…0101   (5)
$t1 = 00…1000   (8)
and   ──────────
$t2 = 00…0000   (0)
```

No coinciden en ningún bit a 1, así que **$t2 = 0**.

EJERCICIO · ¿PARA QUÉ SIRVE UN AND CON 0X007FFFFF?

```
li  $t0, 5
li  $t1, 0x007FFFFF
and $t2, $t1, $t0
```

SOLUCIÓN

`0x007FFFFF` = 9 ceros seguidos de **23 unos**. Al hacer AND, los bits donde hay 0 se borran y los bits donde hay 1 se quedan como estaban. Así que sirve para **quedarse con los 23 bits menos significativos** (aquí $t2 = 5, porque 5 ya cabía en esos bits).

Una constante que se usa para seleccionar bits se llama **máscara**. (Curiosidad: 23 bits es justo la mantisa de un float IEEE 754, así que esta máscara sirve para sacarla.)

TRUCO · LAS TRES MÁSCARAS

AND con máscara → **pone a 0** los bits donde la máscara tiene 0 (selecciona). **OR** con máscara → **pone a 1** los bits donde la máscara tiene 1. **XOR** con máscara → **invierte** los bits donde la máscara tiene 1.

ANALOGÍA

Una máscara AND es como una **plantilla de cartulina con agujeros** encima de un folio: solo ves (te quedas con) lo que hay debajo de los agujeros (los 1).

3.4 Desplazamientos

Mueven los bits de un registro a la izquierda o a la derecha *shamt* posiciones (*shift amount*).

Instrucción	Qué hace	Ejemplo
`sll $t0, $t1, n`	Lógico a la **izquierda**; entran ceros por la derecha. **Equivale a multiplicar por 2^n**	5 << 2 = 0101 → 10100 = **20**
`srl $t0, $t1, n`	Lógico a la **derecha**; entran **ceros** por la izquierda. Divide entre 2^n números sin signo	20 >> 2 = **5**
`sra $t0, $t1, n`	Aritmético a la derecha; entra **copia del bit de signo**. Divide entre 2^n números con signo	-16 >> 2 = **-4**

¡OJO!

-16 = `0xFFFFFFF0`. Con `sra` 2 → `0xFFFFFFFC` = -4 ✓. Con `srl` 2 → `0x3FFFFFFC` = 1 073 741 820 × (han entrado ceros y se ha perdido el signo). Para números negativos, usa `sra`.

TRUCO

Multiplicar por 4 un índice (para recorrer un vector de enteros) se hace muy rápido con `sll $t4, $t2, 2`. Lo usaremos en el capítulo 5.

3.5 Comparaciones

Comparan dos valores y ponen el destino a **1 si se cumple** la condición y a **0 si no**. Su nombre viene de "*set if…*" (activa si…).

Instrucción	Nombre	Equivale a	Tipo
`slt $t0,$t1,$t2`	Set if less than	if ($t1 < $t2) $t0 = 1; else $t0 = 0	R
`slti $t0,$t1,inm`	Set if less than immediate	if ($t1 < inm) $t0 = 1; else 0	I
`seq $t0,$t1,$t2`	Set if equal	if ($t1 == $t2) …	pseudo
`sne $t0,$t1,$t2`	Set if not equal	if ($t1 != $t2) …	pseudo
`sge $t0,$t1,$t2`	Set if greater or equal	if ($t1 >= $t2) …	pseudo
`sgt $t0,$t1,$t2`	Set if greater than	if ($t1 > $t2) …	pseudo
`sle $t0,$t1,$t2`	Set if less or equal	if ($t1 <= $t2) …	pseudo

En las diapositivas aparece `sneq` ; el nombre que acepta el simulador y que viene en el cuadro resumen es `sne`.

EJEMPLO

```
li  $t1, 3
li  $t2, 9
slt $t0, $t1, $t2    # 3 < 9  → $t0 = 1
sge $t3, $t1, $t2    # 3 >= 9 → $t3 = 0
slti $t4, $t1, -15   # 3 < -15 → $t4 = 0
```

RESUMEN DEL CAPÍTULO 3

Destino primero · `i` al final = inmediato · `u` al final = sin signo (sin aviso de overflow) · mult/div reales usan HI (resto / parte alta) y LO (cociente / parte baja) · máscaras: AND borra, OR enciende, XOR invierte · `sll` multiplica por 2^n · `sra` conserva el signo · las `s…` de comparación dejan un 1 o un 0.

4 La memoria: bytes, palabras y vectores

4.1 Cómo está organizada la memoria

La memoria principal es como un **vector gigante de una dimensión**: cada casilla tiene un número, su **dirección** (el índice del vector). Hay dos formas de organizarla:

POR BYTES

Cada casilla es **1 byte** (8 bits). Para traer una palabra hay que transferir 4 bytes. **Es lo que usa MIPS.**

POR PALABRAS

Cada casilla es **1 palabra** (32 bits). Leer un byte supone traer la palabra entera y quedarse con un trozo.

Recuerda: **todo** (instrucciones y datos) está en memoria, así que **todo tiene una dirección**. Y para ejecutarse, un programa tiene que estar cargado en memoria junto con sus datos.

4.2 Instrucciones de acceso a memoria

MIPS es RISC: la ALU no toca la memoria. Solo hay dos tipos de instrucciones que lo hacen: las de **carga** (memoria → registro, empiezan por **l** de *load*) y las de **almacenamiento** (registro → memoria, empiezan por **s** de *store*).

Instrucción	Qué hace	Tamaño
`lw`	Carga una **palabra** de memoria en un registro	4 bytes
`lh` / `lhu`	Carga media palabra con / sin extensión de signo	2 bytes
`lb`	Carga un **byte** y **extiende el signo**	1 byte
`lbu`	Carga un byte **sin** extender el signo (rellena con ceros)	1 byte
`sw`	Guarda un registro entero en memoria	4 bytes
`sh`	Guarda los 16 bits bajos del registro	2 bytes
`sb`	Guarda el byte **menos significativo** del registro	1 byte
`la`	¡No accede a memoria! Carga la **dirección** en el registro	—

♥ Cómo escribir la dirección

Todas tienen la forma `instr registro, dirección`, y la dirección se puede escribir así:

Forma	Ejemplo	Significado
número	`lbu $t0, 0x0F000002`	el byte de la dirección 0x0F000002 (**directo**)
etiqueta	`lbu $t0, etiqueta`	el byte de la dirección que representa la etiqueta (**directo**)
(registro)	`lbu $t0, ($t1)`	el byte cuya dirección está **guardada en $t1** (**indirecto de registro**)
num(registro)	`lbu $t0, 80($t1)`	el byte de la dirección **$t1 + 80** (**relativo a registro**)
etiqueta + num	`lw $t0, vec+8`	la dirección de la etiqueta más 8
etiqueta(registro)	`lw $t0, vec($t1)`	la dirección de la etiqueta más el contenido de $t1

EJEMPLO · LAS TRES FORMAS CON $T1 = 0XFFEE0002

`lbu $a0, 0xFFEE0000` → lee el byte de 0xFFEE0000. `lbu $a0, ($t1)` → lee el byte de 0xFFEE0002. `lbu $a0, 2($t1)` → lee el byte de 0xFFEE0002 + 2 = **0xFFEE0004**.

ANALOGÍA

Directo: "el paquete está en la taquilla 25". **Indirecto de registro**: "mira el papel que tengo en el bolsillo; ahí pone el número de la taquilla". **Relativo**: "es 2 taquillas más allá de la del papel".

4.3 Bytes: lb, lbu y la extensión de signo

Un registro tiene 32 bits pero un byte solo 8. Al cargar un byte en un registro, ¿qué ponemos en los otros 24 bits? Hay dos opciones:

- **lbu** (sin signo): los 24 bits de arriba se ponen **a 0**.
- **lb** (con signo): se hace **extensión de signo**: se **copia el bit 7** del byte en los 24 bits de arriba. Si el byte empezaba por 0 → se rellena con 0; si empezaba por 1 → se rellena con 1.

EJEMPLO 1 · LB $T1, 0X6 CON UN NÚMERO POSITIVO

En la dirección `0x00000006` está `00001001` (9). Como empieza por 0:

$t1 | 00000000 | 00000000 | 00000000 | **00001001** | = 9

EJEMPLO 2 · LB $T1, 0X6 CON UN NÚMERO NEGATIVO

Ahora en `0x00000006` está `11111101` (-3 en complemento a 2). Empieza por 1, así que se rellena con unos:

$t1 | 11111111 | 11111111 | 11111111 | **11111101** | = -3 ✓

lb mantiene el signo: el registro sigue valiendo -3.

¡OJO! EL PROBLEMA DE LOS CARACTERES

La **ñ** tiene código ASCII extendido **164** = `10100100`. Si la cargas con `lb`:

`$t1`	11111111	11111111	11111111	**10100100**

= -92 ×

¡El registro ya no vale 164! Por eso, **para caracteres (y cualquier valor sin signo) usa `lbu`**, que daría `00…0010100100` = 164 ✓.

4.4 Palabras: lw y sw

Una palabra son 32 bits = **4 bytes**, y se guarda en **4 posiciones consecutivas** de memoria que empiezan en una **dirección múltiplo de 4** (0, 4, 8, 12…). Para acceder a ella se da la dirección de su **primer byte**.

Dentro de la palabra, el **byte3** (bits 31–24) es el **más significativo** y el **byte0** (bits 7–0) el **menos significativo**. Pero… ¿en qué orden se colocan los 4 bytes en memoria? Ahí entra el *endianness*.

4.5 Big endian y little endian

DEFINICIÓN

Big endian: en la dirección más pequeña (A) va el byte **más significativo** ("empieza por el extremo grande").

Little endian: en la dirección más pequeña (A) va el byte **menos significativo** ("empieza por el extremo pequeño").

Algunos procesadores (MIPS entre ellos) pueden funcionar de las dos formas: son **bi-endian**.

EJEMPLO · EL NÚMERO 27

$27_{(10} = 11011_{(2}$ = `00000000 00000000 00000000 00011011` = `0x0000001B`

Dir.	Big endian
A	00000000
A+1	00000000
A+2	00000000
A+3	**00011011**

Dir.	Little endian
A	**00011011**
A+1	00000000
A+2	00000000
A+3	00000000

ANALOGÍA

Es como escribir una fecha: **2026-10-21** (lo más importante primero, big endian) o **21-10-2026** (lo menos importante primero, little endian). Los dos son correctos, pero si uno escribe de una forma y otro lee de la otra… lío.

¡OJO! COMUNICACIÓN ENTRE COMPUTADORES DISTINTOS

Si un computador big endian envía por la red sus 4 bytes (00, 00, 00, 1B) tal cual y otro little endian los copia en el mismo orden, este último lee el número `00011011 00000000 00000000 00000000` = `0x1B000000` = 452 984 832, **que no es el 27**. Por eso los protocolos de red fijan un orden de bytes.

EJEMPLO · LW $T1, 0X4 EN CADA TIPO DE MÁQUINA

En memoria, a partir de la dirección 4, están los bytes `00000000, 00000000, 00000001, 00001101`. La dirección de la palabra es la de su **primer byte**: `0x00000004`.

Dir.	Byte
0x4	00000000
0x5	00000000
0x6	00000001
0x7	00001101

Big endian (el primero es el más significativo):

$t1 | 00000000 | 00000000 | 00000001 | 00001101 | = 0x0000010D = **269**

Little endian (el primero es el menos significativo, se "dan la vuelta"):

$t1 | 00001101 | 00000001 | 00000000 | 00000000 | = 0x0D010000

EJEMPLO · ENDIAN.S

```
.data
    b1: .byte 0x00, 0x11, 0x22, 0x33   # 4 bytes seguidos
.text
.globl main
main:
    lw $t0, b1       # ¿qué palabra se lee?
```

En una máquina **big endian**: `$t0 = 0x00112233`. En una **little endian** (como el simulador en un PC): `$t0 = 0x33221100`. Este programa sirve justo para **averiguar** de qué tipo es tu máquina.

4.6 Diferencias entre lw, lb, lbu y la

Con esta memoria (máquina **big endian**):

Dirección	Contenido
0x0F000000	0xCB
0x0F000001	0x12
0x0F000002	0x08
0x0F000003	0x02

Instrucción	$t1 queda	Por qué
`lw $t1, 0x0F000000`	`0xCB120802`	Copia la **palabra** (los 4 bytes) que hay en esa dirección
`lbu $t1, 0x0F000002`	`0x00000008`	Copia el **byte** 0x08, sin extensión de signo
`lb $t1, 0x0F000000`	`0xFFFFFFCB`	0xCB = 11001011 empieza por 1 → se extiende con unos
`la $t1, 0x0F000000`	`0x0F000000`	Copia **la dirección**, no el contenido

TRUCO

`la` = **l**oad **a**ddress = "apúntate la dirección de la taquilla". `lw` = "abre la taquilla y coge lo que hay dentro".

4.7 Escribir en memoria: sw y sb

EJEMPLO PASO A PASO (BIG ENDIAN)

```
li $t1, 18              # 18 = 0x12 = 00010010
li $t2, 24              # 24 = 0x18 = 00011000
sw $t1, 0x0F000000      # escribe la palabra completa de $t1
sw $t2, 0x0F000004      # escribe la palabra completa de $t2
```

Dirección	Tras los sw
0x0F000000	00000000
0x0F000001	00000000
0x0F000002	00000000
0x0F000003	**00010010**
0x0F000004	00000000
0x0F000005	00000000
0x0F000006	00000000
0x0F000007	**00011000**

Dirección	Si luego: sb $t1, 0x0F000001
0x0F000001	**00010010**

`sb` escribe **solo el byte menos significativo** del registro (aquí 00010010) en la dirección indicada.

Resumen: `sw $t0, 0x0F000000` copia la palabra de $t0 en esa dirección; `sb $t0, 0x0F000000` copia solo su byte bajo.

4.8 Alineación

DEFINICIÓN

Un dato que ocupa **K bytes** está **alineado** si la dirección D con la que se accede cumple **D mod K = 0** (D es múltiplo de K).

- Datos de **2 bytes** → en direcciones **pares**.
- Datos de **4 bytes** (int, palabra) → en direcciones **múltiplo de 4**.

- Datos de **8 bytes** (double) → en direcciones **múltiplo de 8**.

Por lo general los computadores **no permiten** acceder a datos no alineados. El objetivo es **minimizar el número de accesos a memoria**: una palabra alineada se trae de una vez. El **compilador** (o el ensamblador) se encarga de colocar los datos en direcciones adecuadas. Algunas arquitecturas, como **Intel**, sí permiten datos no alineados, pero entonces leer uno cuesta **varios accesos**.

EJEMPLO

```
.data
    b1: .byte 0x0f      # ocupa la dirección 0
    w1: .word 10        # NO puede ir en la 1: el ensamblador la lleva a la 4
```

Si la palabra empezara en la dirección 1 (bytes 1, 2, 3 y 4) estaría repartida entre dos "filas" de la memoria y harían falta dos accesos. Para forzar la alineación a mano se usa `.align 2` (múltiplo de $2^2 = 4$).

ANALOGÍA

La memoria se lee por "filas" de 4 bytes, como un tren con vagones de 4 asientos. Si tu grupo de 4 amigos se sienta en un mismo vagón (alineado), el revisor los atiende de una vez. Si se reparten entre dos vagones, tiene que pasar por los dos.

4.9 Tipos de datos en ensamblador

En ensamblador no hay "tipos" como en C: solo bytes. Somos nosotros los que decidimos cómo interpretarlos: booleanos caracteres enteros reales vectores cadenas de caracteres matrices otras estructuras.

♥ Vectores

Un vector es un conjunto de elementos **colocados uno detrás de otro** en memoria. Si el vector empieza en *Direccion_inicio* y cada elemento ocupa *p* bytes:

FÓRMULA CLAVE

dirección de v[j] = Direccion_inicio + j · p

Para `int` (4 bytes): v[j] está en inicio + 4j · Para `char` (1 byte): en inicio + j

EJEMPLO · INT VEC[5]; VEC[4] = 8; (TRES FORMAS)

```
.data
    .align 2            # siguiente dato alineado a 4
    vec: .space 20       # 5 elementos * 4 bytes
```

vec[4] está en vec + 4·4 = **vec + 16**.

A) relativo a registro base

```
la $t1, vec
li $t2, 8
sw $t2, 16($t1)
```

B) sumando la dirección a mano

```
li  $t0, 16
la  $t1, vec
add $t3, $t1, $t0
li  $t2, 8
sw  $t2, ($t3)
```

C) relativo a registro índice

```
li $t2, 8
li $t1, 16
sw $t2, vec($t1)
```

Las tres hacen lo mismo. En A el registro tiene la **base** (dónde empieza el vector) y el número es el desplazamiento; en C es al revés: la etiqueta da la base y el registro el **índice** (desplazamiento).

EJEMPLO · V[3] = 5; V[4] = 8;

```
.data
    .align 2
    v: .space 20          # 5 int * 4 bytes/int
.text
.globl main
main:
    la $t0, v             # $t0 = dirección de v
    li $t1, 5
    sw $t1, 12($t0)       # v[3]: 3*4 = 12

    li $t0, 16            # desplazamiento de v[4]
    li $t1, 8
    sw $t1, v($t0)        # v[4]
```

EJERCICIO

Si V es un array de enteros (int) y V representa la dirección de inicio del vector: a) ¿En qué dirección está V[5]? b) ¿Qué instrucciones cargan en `$t0` el valor de V[5]?

SOLUCIÓN

a) **V + 5·4 = V + 20**.

b) `li $t1, 20` y después `lw $t0, V($t1)`. (O directamente `la $t1, V` + `lw $t0, 20($t1)`.)

EJEMPLOS DEL LIBRO · VECTORES

Las variables g, h, i están en `$14, $15, $16` y el array A empieza en la posición `Inicio`. Recuerda: `lw $1, Inicio($2)` trae lo que hay en (Inicio + $2) al registro $1 y `sw $1, Inicio($2)` lleva $1 a esa posición.

g = h + A[i];

```
lw  $8, Inicio($16)  # $8 = A[i]
add $14, $15, $8     # g = h + A[i]
```

A[i] = h + A[i];

```
lw  $8, Inicio($16)  # $8 = A[i]
add $8, $15, $8      # $8 = h + A[i]
sw  $8, Inicio($16)  # A[i] = $8
```

¡OJO!

Esto solo funciona si `$16` ya contiene **i·4** (el desplazamiento en bytes). Si contiene i "a secas", antes hay que hacer `sll $9, $16, 2` (i·4) y usar `Inicio($9)`. En MIPS la memoria va por bytes, ¡no te olvides del ×4!

♥ Cadenas de caracteres

Una cadena es un vector de **bytes** (un carácter = 1 byte) terminado en un **0** (con `.asciiz`). Se recorre con `lbu` avanzando de 1 en 1.

EJEMPLO · LONGITUD DE UNA CADENA (STRLEN)

```
.data
    cad: .asciiz "hola"
.text
main:   la   $t0, cad          # puntero al carácter actual
         li   $t1, 0            # contador
bucle:  lbu  $t2, 0($t0)       # carácter actual
         beqz $t2, fin          # si es el 0 final, terminamos
         addi $t1, $t1, 1       # un carácter más
         addi $t0, $t0, 1       # siguiente byte
         b    bucle
fin:    # aquí $t1 = 4
```

RESUMEN DEL CAPÍTULO 4

Memoria por bytes · palabra = 4 bytes en dirección múltiplo de 4 · **lb** extiende signo, **lbu** rellena con ceros (úsalo con caracteres) · **sb** guarda el byte bajo · **la** carga la dirección, no el contenido · big endian = el más significativo en la dirección baja; little endian = al revés · alineado ⇔ D mod K = 0 · v[j] está en inicio + j·tamaño.

5 Control de flujo: if, else, while, switch

IDEA CLAVE

Normalmente el PC avanza de 4 en 4 y las instrucciones se ejecutan en orden. Las **instrucciones de bifurcación (salto)** cambian el PC para seguir por otro sitio. *"Lo que distingue a un computador de una simple calculadora es su capacidad para **tomar decisiones**."*

Hay tres tipos:

Tipo	Qué hace	Ejemplo
Salto condicional	Salta a una etiqueta **solo si** se cumple una condición; si no, sigue con la siguiente	`bne $t0, $t1, etiqueta`
Salto incondicional	Salta **siempre**	`j etiqueta`, `b etiqueta`
Llamada a procedimiento	Salta a una subrutina **recordando dónde volver**	`jal subrutina` … `jr $ra`

5.1 Saltos condicionales

Instrucción	Salta a etiq si…	Tipo
`beq $t0, $t1, etiq`	$t0 == $t1 (*branch if equal*)	I ★
`bne $t0, $t1, etiq`	$t0 != $t1 (*branch if not equal*)	I ★
`beqz $t1, etiq`	$t1 == 0	pseudo
`bnez $t1, etiq`	$t1 != 0	pseudo
`bgt $t0, $t1, etiq`	$t0 > $t1	pseudo
`bge $t0, $t1, etiq`	$t0 >= $t1	pseudo
`blt $t0, $t1, etiq`	$t0 < $t1	pseudo
`ble $t0, $t1, etiq`	$t0 <= $t1	pseudo
`bgez $t0, etiq`	$t0 >= 0	I ★
`bgtz $t0, etiq`	$t0 > 0	I ★
`blez $t0, etiq`	$t0 <= 0	I ★
`bltz $t0, etiq`	$t0 < 0	I ★

★ = instrucción real que tu profe ha marcado en la tabla del libro. Las que comparan con 0 (`bgez, bgtz, blez, bltz`) son reales; las que comparan dos registros con <, >… son pseudoinstrucciones.

TRUCO PARA LOS NOMBRES

b = *branch* (bifurcar) · **eq** = igual · **ne** = distinto · **gt** = mayor (*greater than*) · **ge** = mayor o igual · **lt** = menor (*less than*) · **le** = menor o igual · **z** = comparado con cero.

♥ Cómo se hacen las comparaciones que no existen: slt + bne

Los compiladores de MIPS crean **todas** las condiciones con solo `slt`, `beq`, `bne` y el registro `$0` (que siempre vale 0). Por ejemplo, "salta a Menor si a < b" (con a en `$16` y b en `$17`):

```
slt $1, $16, $17      # $1 = 1 si a < b
bne $1, $0, Menor     # salta a Menor si $1 != 0  (o sea, si a < b)
...
Menor:
```

Así se implementa un "salta si menor" que no existe en MIPS. Se usa `$1` (= `$at`), que está reservado como registro transitorio para el ensamblador: de hecho, así es como el ensamblador traduce `blt`.

5.2 Saltos incondicionales

Instrucción	Qué hace
`j etiqueta`	Salta **siempre** a la etiqueta (`PC ← dirección`). ★
`b etiqueta`	También salta siempre, pero codificando un **desplazamiento** respecto al PC (diferencia en el capítulo 7).
`jr $t0`	Salta a la dirección **guardada en el registro** (*jump register*). ★
`jal etiqueta`	Salta y guarda la dirección de vuelta en `$ra` (capítulo 8).

5.3 Traducir C a ensamblador: las plantillas

TRUCO DE ORO

En ensamblador casi siempre se pone la condición **al revés**: en vez de "si se cumple, haz esto", se escribe "**si NO se cumple, sáltate** esto". Por eso un `if (i == j)` suele convertirse en un `bne`.

♥ if con goto

EJEMPLO DEL LIBRO · F, G, H, I, J EN $16 … $20

```
if (i == j) goto L1;
f = g + h;
L1: f = f - i;
```

```
      beq $19, $20, L1    # ir a L1 si i ==
      add $16, $17, $18   # f = g + h (se sa
L1:   sub $16, $16, $19   # f = f - i (siemp
```

Aquí sí se usa la condición "tal cual", porque el C ya tiene un `goto`.

♥ if – else

EJEMPLO DEL LIBRO

```
if (i == j)
    f = g + h;
else
    f = g - h;
```

```
        bne $19, $20, Else  # ir a Else si i
        add $16, $17, $18   # f = g + h
        j   Exit            # ¡saltar el else
Else:   sub $16, $17, $18   # f = g - h
Exit:
```

El `j Exit` es imprescindible: sin él, después de hacer la parte del *if* se ejecutaría también la del *else*.

ANALOGÍA

Es como un libro de "elige tu propia aventura": "si abres la puerta, ve a la página 40". El `j Exit` es el "ahora ve a la página 50" que te impide leer también el otro camino.

♥ while

EJEMPLO DEL LIBRO · WHILE (SAVE[I] == K) I = I + J;

Datos: `$19 = i`, `$20 = j`, `$21 = k`, `$10 = 4`; el vector `save` empieza en la dirección `Inicio`.

```
Lazo:  mul $9, $19, $10      # $9 = i * 4  (desplazamiento de save[i])
       lw  $8, Inicio($9)    # $8 = save[i]
       bne $8, $21, Exit     # si save[i] != k, salir del bucle
       add $19, $19, $20     # i = i + j
       j   Lazo              # volver a comprobar la condición
Exit:
```

Estructura de cualquier while: **etiqueta arriba → comprobar la condición al revés (si no se cumple, salir) → cuerpo → saltar arriba**.

(Corregido respecto a la diapositiva: allí pone `mult $9,$19,$10` con el comentario "1*4" y `add $19,419,$20`; son erratas de `i*4` y `$19`. Además `mult` real solo tiene dos operandos, por eso uso `mul`.)

♥ for (recorrer un vector)

EJEMPLO · SUMAR LOS ELEMENTOS DE UN VECTOR

```
int v[5] = {3,7,1,9,5};
int n = 5, suma = 0;
for (i = 0; i < n; i++)
    suma = suma + v[i];
printf("%d", suma);
```

```
.data
v:     .word 3, 7, 1, 9, 5
n:     .word 5
.text
main:  la   $t0, v           # inicio del vector
       lw   $t1, n           # n
       li   $t2, 0           # i = 0
       li   $t3, 0           # suma = 0
bucle: bge  $t2, $t1, fin    # si i >= n, salir
       sll  $t4, $t2, 2      # $t4 = i*4
       add  $t4, $t0, $t4    # $t4 = dirección de v
       lw   $t5, 0($t4)      # $t5 = v[i]
       add  $t3, $t3, $t5    # suma += v[i]
       addi $t2, $t2, 1      # i++
       b    bucle
fin:   move $a0, $t3         # imprimir suma
       li   $v0, 1
       syscall               # → 25
       li   $v0, 10
       syscall
```

♥ switch con jr (bifurcar a registro)

EJEMPLO DEL LIBRO · SWITCH (K)

Idea: se guarda en memoria una **tabla de direcciones** (una por cada caso), se lee la que toca según k y se salta con `jr`. Datos: `$16 = f`, `$17 = g`, `$18 = h`, `$19 = i`, `$20 = j`, `$21 = k`, `$10 = 4`.

```
switch (k) {
  case 0: f = i + j; break;
  case 1: f = g + h; break;
  case 2: f = g - h; break;
}
```

```
.data
Tabla: .word L0, L1, L2      # direcciones de cada caso
.text
       mul $9, $21, $10        # $9 = k*4
       lw  $8, Tabla($9)       # $8 = Tabla[k]
       jr  $8                  # bifurca según $8
L0:    add $16, $19, $20       # k = 0: f = i + j
       j   Salir
L1:    add $16, $17, $18       # k = 1: f = g + h
       j   Salir
L2:    sub $16, $17, $18       # k = 2: f = g - h
Salir:
```

Cada `j Salir` hace el papel del `break`. (Corregido: la diapositiva multiplica `$19` (i) en vez de `$21` (k).)

♥ Un bucle más de las diapositivas

EJEMPLO · ¿QUÉ CALCULA?

```
        li   $t0, 8
        li   $t1, 4
        li   $t2, 1
        li   $t4, 0
while:  bge  $t4, $t1, fin     # mientras $t4 < 4
        mul  $t2, $t2, $t0     #   $t2 = $t2 * 8
        addi $t4, $t4, 1       #   $t4++
        b    while
fin:    move $t2, $t4
```

El bucle da 4 vueltas y calcula $t2 = 8^4 = **4096**. Después, `move $t2, $t4` sobreescribe $t2 con el contador (4). Este programa lo usaremos en el capítulo 7 para ver cómo se codifican las etiquetas.

RESUMEN DEL CAPÍTULO 5

if → condición al revés + saltar el bloque · if-else → además un `j Exit` al final del bloque if · while/for → etiqueta arriba, condición de salida, cuerpo, `j` / `b` arriba · switch → tabla de direcciones + `jr` · `slt` + `bne` con `$0` construyen cualquier comparación.

6 Formato de las instrucciones

6.1 Campos de una instrucción

Una instrucción máquina es **autocontenida**: incluye el código de operación, dónde están los operandos, dónde va el resultado, cuál es la siguiente instrucción y cómo se representan los operandos. Para organizar todo eso, se divide en **campos**.

- El tamaño de la instrucción se ajusta al de la **palabra** (o a un múltiplo). En MIPS **todas** ocupan una palabra: 32 bits.
- En los campos se codifica la **operación** (código de operación, que también indica el formato) y los **operandos**: dónde están, dónde va el resultado y, si es un salto, dónde está la siguiente instrucción. Eso último puede ser **implícito** (`PC ← PC + 4`) o **explícito** (`j 0x01004` modifica el PC).
- Puede haber operandos **implícitos** (que no aparecen escritos, como el 0 de `beqz`).
- El tamaño de un campo **limita lo que puede codificar**: un campo de n bits codifica 2^n valores distintos.

DEFINICIÓN

El **formato** de una instrucción especifica el significado de cada uno de sus bits: cuántos campos hay, qué tamaño tienen y qué significan. La **longitud del formato** es el número de bits de la instrucción.

Una arquitectura suele ofrecer **pocos formatos**, para que la unidad de control sea sencilla. Además se usan de forma **sistemática**: los campos del mismo tipo tienen siempre la misma longitud y el formato se elige mediante el **código de operación**, que suele ser el **primer campo**.

ANALOGÍA

Un formato es como un **formulario** con casillas de tamaño fijo: "Nombre (20 letras) · DNI (9) · Edad (3)". Si la casilla de edad tiene 3 huecos, no puedes escribir una edad de 4 cifras. Igual con los bits: un campo de 5 bits solo puede guardar números del 0 al 31.

6.2 Los tres formatos del MIPS: R, I y J

Tipo R (registro) · aritméticas y lógicas entre registros

op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

Tipo I (inmediato) · transferencias con memoria, inmediatos y saltos condicionales

op	rs	rt	inmediato / dirección / desplazamiento
6 bits	5 bits	5 bits	16 bits

Tipo J (salto) · j y jal

op 6 bits	dirección de destino 26 bits

Campo	Significado
op	Código de operación de la instrucción
rs	Primer registro del operando fuente
rt	Segundo registro fuente (en tipo I suele ser el **destino**)
rd	Registro del operando destino
shamt	Cantidad de desplazamiento (*shift amount*, para sll/srl/sra)
funct	Función: variante concreta de la operación

¿Por qué 5 bits para los registros? Porque hay 32 registros y $2^5 = 32$.

UN BUEN DISEÑO EXIGE COMPROMISOS

Los diseñadores de MIPS quisieron que **todas** las instrucciones tuvieran la **misma longitud**, así que hicieron falta **varios formatos**. Para no complicar el hardware, los formatos se parecen: los tres primeros campos de R e I son iguales (op, rs, rt) y el cuarto campo del tipo I mide lo mismo que los tres últimos del tipo R juntos (5+5+6 = 16). El hardware sabe si tratar esa última mitad como tres campos (tipo R) o como uno solo (tipo I) **mirando el primer campo (op)**.

6.3 Codificar instrucciones en binario

EJEMPLO 1 · ADD $8, $9, $10 (TIPO R)

Significa $8 ← $9 + $10, así que rs = 9, rt = 10, rd = 8. Para las aritméticas op = 0, shamt = 0 y la función suma es funct = 32.

op	rs	rt	rd	shamt	funct
0	9	10	8	0	32
`000000`	`01001`	`01010`	`01000`	`00000`	`100000`

Todo junto: `000000 01001 01010 01000 00000 100000` = **0x012A4020**.

EJEMPLO 2 · ADDI $21, $22, -50 (TIPO I)

$21 ← $22 + (-50), así que rs = 22 (fuente), rt = 21 (destino). op de addi = 8. El inmediato -50 va en **complemento a 2 con 16 bits**: 50 = 0000000000110010 → invertir y sumar 1 → **1111111111001110**.

op	rs	rt	inmediato
8	22	21	-50
`001000`	`10110`	`10101`	`1111111111001110`

Todo junto: `001000 10110 10101 1111111111001110` = **0x22D5FFCE**.

¡OJO CON EL ORDEN!

En ensamblador se escribe el destino primero (`add rd, rs, rt`; `addi rt, rs, inm`), pero en binario el orden de los campos es **op, rs, rt, rd…**. ¡No los copies en el orden en que aparecen escritos!

♥ ¿Y si el inmediato no cabe en 16 bits?

`addi $t0, $t0, 0xABABCDCD` no se puede codificar en 32 bits (solo el inmediato ya ocupa 32). El ensamblador lo acepta pero lo traduce a tres instrucciones usando `$at`, el registro reservado para él por convenio:

```
lui $at, 0xABAB          # $at = 0xABAB0000
ori $at, $at, 0xCDCD     # $at = 0xABABCDCD
add $t0, $t0, $at        # la suma de verdad
```

6.4 Longitud del formato y código de operación

Alternativa	Cómo es	Ejemplos
Longitud única	Todas las instrucciones miden lo mismo	MIPS32 (32 bits), PowerPC (32 bits)
Longitud variable	Cada instrucción mide distinto. ¿Cómo sabe la CPU cuánto mide? **Por el código de operación**	IA32 (Intel): número variable de bytes

♥ El código de operación

- **Tamaño fijo**: con **n bits** → $\mathbf{2^n}$ códigos. Si necesitas **m** códigos → $\mathbf{\lceil \log_2 m \rceil}$ **bits** (redondeando hacia arriba).
- **Campos de extensión**: si no hay bastantes códigos, se usa un campo extra. En MIPS, todas las aritmético-lógicas tipo R tienen **op = 0**, y la operación concreta va en **funct**.
- **Tamaño variable**: las instrucciones **más frecuentes** llevan los códigos **más cortos** (como en Morse, donde la E, la letra más usada, es un solo punto).

EJERCICIO · PREGUNTAS SOBRE LA UNIDAD DE CONTROL

1) ¿Cómo sabe la unidad de control el formato de la instrucción que está ejecutando? 2) ¿Y el número de operandos? 3) ¿Y el formato de cada operación?

SOLUCIÓN

Las tres respuestas son la misma: **por el código de operación**, que está en el primer campo y es lo primero que se decodifica. Cada código lleva asociado un formato (y con él, cuántos operandos hay, de qué tipo y dónde está cada campo). En MIPS, si op = 0 se mira además el campo **funct** para saber la operación exacta.

6.5 Ejercicios de diseño de formatos

RECETA PARA ESTOS EJERCICIOS

1) El tamaño de palabra suele marcar el tamaño de la instrucción. 2) Bits de código de operación = $\lceil \log_2(\text{nº instrucciones}) \rceil$. 3) Bits por registro = $\lceil \log_2(\text{nº registros}) \rceil$. 4) Bits por dirección de memoria = ancho de las direcciones (si direcciona por bytes y el computador es de n bits, normalmente n). 5) Suma todo; si sobra, es relleno; si no cabe en una palabra, usa varias.

EJERCICIO 1 · ADDX R1, R2, R3

Computador con palabra de **16 bits**, **60 instrucciones** y **8 registros**. Indica el formato de `ADDx R1 R2 R3` (los tres son registros).

SOLUCIÓN

- La palabra de 16 bits define el tamaño de la instrucción: **16 bits**.
- 60 instrucciones → $2^5 = 32$ no llega, $2^6 = 64$ sí → **6 bits** de código de operación.
- 8 registros → $2^3 = 8$ → **3 bits** por registro, × 3 registros = 9 bits.
- Sobra 16 - 6 - 3 - 3 - 3 = **1 bit**, que se usa de relleno.

Cód. op.	R1	R2	R3	–
6 bits	3	3	3	1

EJERCICIO 2 · ADDV R1, R2, M

Computador de **16 bits** que direcciona la memoria **por bytes**, con **60 instrucciones** y **8 registros**. Indica el formato de `ADDV R1, R2, M` (R1 y R2 registros, M una dirección de memoria).

SOLUCIÓN

- Código de operación: 60 instrucciones → **6 bits**.
- R1 y R2: 8 registros → **3 bits** cada uno.
- M: el computador es de 16 bits, así que sus direcciones son de **16 bits** (2^{16} bytes = 64 KB).
- Total: 6 + 3 + 3 + 16 = **28 bits**, que **no caben** en una palabra de 16. Hacen falta **2 palabras (32 bits)**, y sobran 4 bits de relleno.

op	R1	R2	relleno	M
6	3	3	4	16 bits (2ª palabra)

El relleno también podría ir al final; lo habitual es dejar la dirección sola en la segunda palabra para leerla entera de una vez.

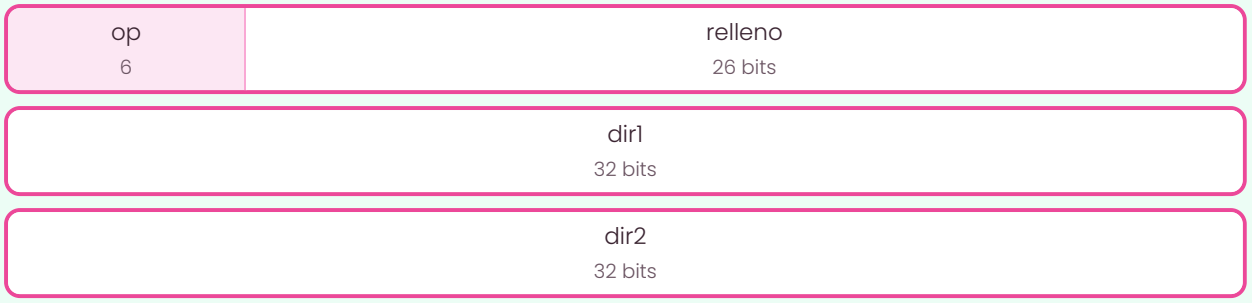
EJERCICIO 3 · SWAPM DIR1, DIR2

Computador de **32 bits**, direccionamiento **por bytes**, **64 instrucciones** y **128 registros**. `SWAPM dir1, dir2` intercambia el contenido de las posiciones de memoria dir1 y dir2. Se pide: a) espacio de memoria direccionable; b) formato de la instrucción; c) fragmento equivalente en ensamblador del MIPS 32; d) si se obliga a que quepa en una palabra, ¿qué rango de direcciones se podría usar (en binario puro)?

SOLUCIÓN

a) Direcciones de 32 bits y una dirección por byte → 2^{32} bytes = **4 GB**.

b) 64 instrucciones → 2^6 → **6 bits** de código. Cada dirección: **32 bits**. Total 6 + 32 + 32 = **70 bits** → no cabe en una palabra; hacen falta **3 palabras (96 bits)**, sobrando 26 bits de relleno. (Los 128 registros no se usan en esta instrucción: es una trampa del enunciado.)



c) MIPS no puede operar memoria con memoria, así que se pasa por registros:

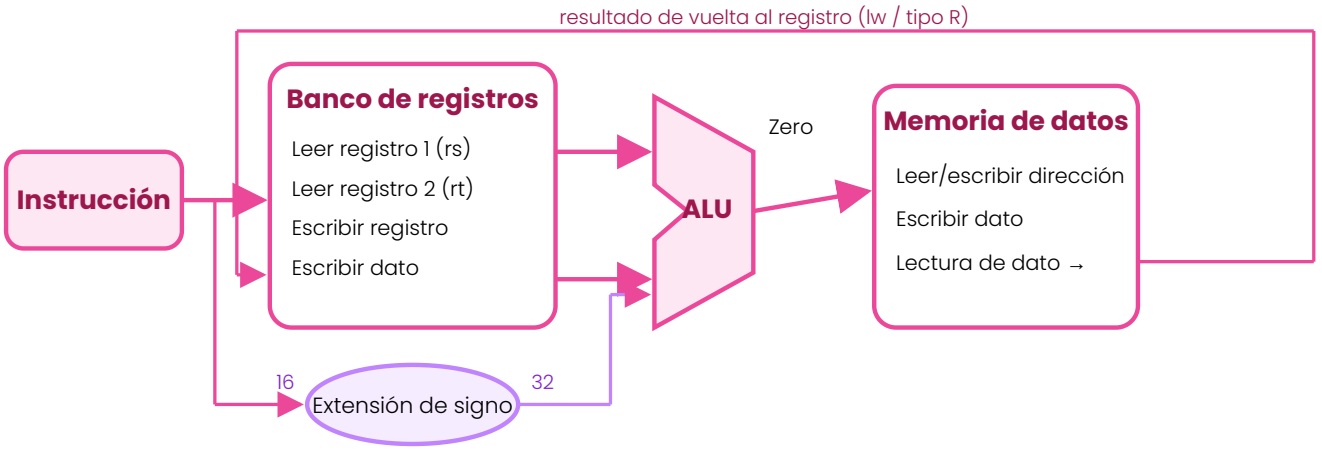
```
lw $t0, dir1      # $t0 = Mem[dir1]
lw $t1, dir2      # $t1 = Mem[dir2]
sw $t0, dir2      # Mem[dir2] = antiguo Mem[dir1]
sw $t1, dir1      # Mem[dir1] = antiguo Mem[dir2]
```

d) En una palabra de 32 bits: 32 - 6 = 26 bits para dos direcciones → **13 bits** cada una → direcciones de **0 a 2^{13} - 1 = 8191** (0x0000 a 0x1FFF), es decir, solo los primeros 8 KB de memoria.



6.6 El camino de datos (cómo se ejecuta por dentro)

Las diapositivas del libro muestran qué partes del hardware usa cada formato. El esquema general es este:



Instrucción	Qué partes usa
Tipo R `add $t0,$t1,$t2`	Tiene **tres registros**: hay que **leer dos** del banco de registros y **escribir uno**. Para leer: una entrada que diga qué registro leer (5 bits, porque $2^5 = 32$) y una salida con su valor. Para escribir: el número del registro y el dato. Las escrituras las controla una **señal de control de escritura**, que debe estar activa cuando llega el reloj. Las entradas y salidas de datos son de 32 bits. La ALU está controlada por una señal de 3 bits que dice qué operación hacer.

Carga/almac.
`lw $1, desp($2)`
`sw $1, desp($2)`
	Calculan la dirección sumando el registro base (`$2`) con el campo de desplazamiento de **16 bits**, que se pasa a 32 bits con la unidad de **extensión de signo**. En un **sw** también se lee del banco el valor a guardar (`$1`) y se escribe en la memoria de datos. En un **lw**, el dato leído de memoria se escribe en el registro (`$1`).

Salto `beq $1, $2, desp`	Tres operandos: dos registros que se comparan (la ALU los resta y mira la salida **Zero**) y un desplazamiento de 16 bits. La dirección de destino = **(PC + 4) + desplazamiento** con el signo extendido y **desplazado 2 a la izquierda** (×4, porque cuenta instrucciones de 4 bytes). Si son iguales, esa dirección pasa a ser el nuevo PC; si no, el PC sigue siendo PC + 4, como en cualquier otra instrucción.

7 Modos de direccionamiento

DEFINICIÓN

El **modo de direccionamiento** es el procedimiento que permite saber **dónde está** un operando, un resultado o una instrucción.

Los operandos pueden estar en cuatro sitios: en la **propia instrucción**, en los **registros** del procesador, en la **memoria principal** o en las **unidades de E/S**. Los modos son:

Implícito · **Inmediato** · **Directo** a registro · a memoria · **Indirecto** a registro · a memoria · **Relativo** a registro índice · a registro base · a PC · a pila

ANALOGÍA GENERAL · ENCONTRAR UN REGALO

Inmediato: te dan el regalo en la mano. **Directo**: "está en el cajón 5". **Indirecto**: "en el cajón 5 hay una nota que dice en qué cajón está" (un paso más). **Relativo**: "está 3 cajones a la derecha del cajón que te diga tu hermana".

7.1 Los modos uno a uno

Modo	Ejemplo	Dónde está el operando	Ventajas / inconvenientes
Implícito	`beqz $a0, etiq`	No está escrito en la instrucción, pero forma parte de ella. Aquí se compara $a0 con **$zero**, que es el operando implícito.	Rápido (no accede a memoria), pero solo posible en pocos casos.
Inmediato	`li $a0, 0x4f51`	**Dentro de la propia instrucción**: el valor 0x00004f51 va codificado en ella.	Rápido. Pero no siempre cabe (16 bits): si es más grande se necesita lui + ori.
Directo a registro (de registro)	`move $a0, $a1`	En un **registro**; en la instrucción va el número del registro.	Muy rápido; pocos registros → pocos bits para codificarlos → instrucciones cortas. Pero hay pocos registros.
Directo a memoria	`lw $t0, 0xFFF0`	En **memoria**, y la **dirección** va en la instrucción. Carga en $t0 la palabra de 0xFFF0.	Mucho espacio (más que el banco de registros), pero memoria es más lenta y las direcciones largas alargan la instrucción.
Indirecto de registro	`lw $a0, ($a1)`	En memoria, en la dirección que **contiene el registro** $a1.	Amplio espacio de direcciones con instrucciones cortas.
Indirecto a memoria	`LD R1, [DIR]` (IEEE 694, no existe en MIPS)	En DIR hay **otra dirección**, y en esa está el operando.	Amplio espacio; puede anidarse (`LD R1,[[[.R1]]]`), pero requiere varios accesos a memoria → lento.
Relativo a registro base	`lw $a0, 12($t1)`	En la dirección **$t1 + 12**. El registro tiene la dirección **base**; el número es el desplazamiento.	Ideal para estructuras y para la pila.
Relativo a registro índice	`lw $a0, dir($t1)`	En la dirección **dir + $t1**. El número es la dirección; el registro es el desplazamiento (**índice**).	Ideal para recorrer vectores.
Relativo a PC	`beqz $a0, etiq`	La etiqueta se codifica como un **desplazamiento** desde la instrucción actual: si $a0 = 0, PC ← PC + desplazamiento.	Código reubicable; alcance limitado.
Relativo a pila	`lw $t0, 8($sp)`	En la dirección **$sp + desplazamiento**.	Para variables locales y datos guardados en la pila.

DIRECTO FRENTE A INDIRECTO

En el **directo** se indica **dónde está el operando** (qué registro o qué posición de memoria). En el **indirecto** se indica **dónde está la dirección del operando**: hay que ir a buscar esa dirección primero, así que se añade un nivel (o varios) de direccionamiento.

¡OJO! BASE FRENTE A ÍNDICE

En la escritura parecen iguales (`número(registro)`). La diferencia es **qué representa cada parte**: si el registro es la dirección de inicio y el número un desplazamiento pequeño → **base** (`12($t1)`). Si el número es una dirección (una etiqueta) y el registro cuánto avanzar → **índice** (`v($t1)`).

7.2 El contador de programa en el MIPS

El **PC** es un registro de **32 bits** que guarda la **dirección de la siguiente instrucción** a ejecutar. Como en MIPS cada instrucción ocupa 32 bits (1 palabra = 4 bytes), la siguiente está 4 bytes después: **PC = PC + 4**.

EJEMPLO · UN PROGRAMA EN MEMORIA

Dirección	Instrucción	Qué hace
`0x00400000`	`or $2, $0, $0`	$2 = 0
`0x00400004`	`slt $8, $0, $5`	$8 = 1 si 0 < $5
`0x00400008`	`beq $8, $0, 3`	si $5 ≤ 0, salta 3 instrucciones → 0x00400018 (fin)
`0x0040000c`	`add $2, $2, $4`	$2 += $4
`0x00400010`	`addi $5, $5, -1`	$5 -= 1
`0x00400014`	`j 0x100001`	vuelve a 0x100001 × 4 = 0x00400004 (el slt)

Este programa calcula **$2 = $4 · $5** sumando $4 tantas veces como indique $5. Fíjate en dos cosas: el `beq` guarda un **desplazamiento en instrucciones** (3) y el `j` guarda la dirección **dividida entre 4** (contada en palabras): 0x100001 · 4 = 0x400004.

7.3 Direccionamiento relativo a PC

`beq $t0, $1, etiqueta` es de tipo I: la etiqueta tiene que caber en el campo de 16 bits. Lo que se guarda es **cuántas instrucciones hay que saltar**, y si se cumple la condición: **PC = PC + (etiqueta · 4)**.

REGLA CLAVE

Cuando se ejecuta una instrucción, **el PC ya apunta a la siguiente**. Así que el desplazamiento se cuenta **desde la instrucción siguiente al salto**:

desplazamiento = (dirección destino - (dirección del salto + 4)) / 4

EJEMPLO · ¿CUÁNTO VALE FIN?

```
bucle:  beq  $t0, $1, fin     # ← al ejecutarse, el PC ya apunta al add
        add  $t8, $t4, $t4    # 1
        addi $t0, $0, -1      # 2
        j    bucle            # 3
fin:    ...                   # → fin == 3
```

EJEMPLO · DESPLAZAMIENTOS EN UN BUCLE (EL DEL CAPÍTULO 5)

Dirección	Ensamblador	Cómo queda codificado
0x0000100	li $t0, 8	li $t0, 8
0x0000104	li $t1, 4	li $t1, 4
0x0000108	li $t2, 1	li $t2, 1
0x000010C	li $t4, 0	li $t4, 0
0x0000110	while: bge $t4, $t1, fin	bge $t4, $t1, **3**
0x0000114	mul $t2, $t2, $t0	mul $t2, $t2, $t0
0x0000118	addi $t4, $t4, 1	addi $t4, $t4, 1
0x000011C	b while	b **−4**
0x0000120	fin: move $t2, $t4	move $t2, $t4

fin: desde el bge, el PC vale 0x114; (0x120 - 0x114)/4 = 12/4 = **3** → PC = PC + 3·4.
while: desde el b, el PC vale 0x120; (0x110 - 0x120)/4 = -16/4 = **-4** → PC = PC + (-4)·4. El desplazamiento es negativo porque saltamos hacia atrás (va en complemento a 2).

Simplificación de las diapositivas: `li` y `bge` son pseudoinstrucciones y en realidad podrían ocupar más de una instrucción, pero para el ejercicio se cuenta cada una como una.

7.4 Diferencia entre b y j

j dirección (tipo J)



Salto **absoluto**: **PC = dirección**.

b desplazamiento (tipo I)



Salto **relativo**: **PC = PC + desplazamiento · 4**. Permite que el código sea **reubicable** en memoria.

ANALOGÍA

`j` es dar una **dirección postal** ("Calle Ancha 7"): si te mudas, ya no vale. `b` es decir "**tres casas más adelante**": funciona estés donde estés. Por eso el código con desplazamientos se puede mover (reubicar) en memoria sin cambiarlo.

EJERCICIO · ¿CUÁL ES MEJOR PARA BUCLES PEQUEÑOS?

Dadas dos instrucciones de salto incondicional: 1) `j etiqueta1` (6 bits de op + 26 de dirección, que se carga en el PC) y 2) `b etiqueta2` (6 + 5 + 5 + 16 bits de desplazamiento en complemento a dos, que se suma al PC). Indica razonadamente cuál es más apropiada para bucles pequeños.

SOLUCIÓN

Ventajas de la opción 1 (j): el cálculo de la dirección es más rápido (solo hay que cargarla, no sumar) y el rango de direcciones es mayor → mejor para saltos lejanos / bucles grandes.

Ventajas de la opción 2 (b): el rango al que puede saltar es menor, pero en un bucle pequeño el salto siempre es cercano, así que basta; y además permite **código reubicable**.

→ La **opción 2** es la más apropiada para bucles pequeños.

Curiosidad: en el MIPS real, los 26 bits de `j` se multiplican por 4 (dan 28 bits) y se completan con los 4 bits altos del PC.

7.5 Resumen de modos en MIPS

Modo	Cómo se escribe	Ejemplo
Inmediato	valor	`li $t1, 4` · `addi $t0,$t0,1`
De registro	$r	`move $a0, $a1`
Directo	dir	`lw $t0, etiqueta`
Indirecto de registro	($r)	`lw $a0, ($a1)`
Relativo a registro	valor($r) — valor puede ser una dirección (registro **índice**) o un desplazamiento (registro **base**)	`lw $t0, 4($a0)`
Relativo a PC	etiqueta en un salto	`beq $t0,$t1,etiqueta`
Relativo a pila	desplazamiento($sp)	`sw $ra, 0($sp)`

EJERCICIO · ¿QUÉ MODOS SE USAN?

1. `li $t1, 4` 2. `lw $t0, 4($a0)` 3. `bnez $a0, etiqueta`

SOLUCIÓN

Cada **operando** tiene su modo:

1. `$t1` → directo a registro · `4` → inmediato.
2. `$t0` → directo a registro · `4($a0)` → relativo a registro base.
3. `$a0` → directo a registro · `etiqueta` → relativo al contador de programa (y el 0 con el que se compara es implícito).

EJEMPLOS · LA ETIQUETA SEGÚN LA INSTRUCCIÓN

Instrucción	Modo de "label"	Por qué
`la $t0, label`	**inmediato**	El operando es una dirección, **pero no se accede a ella**: la propia dirección es el dato.
`lw $t0, label`	**directo a memoria**	Hay que **ir a esa dirección** para obtener el valor.
`bne $t0, $t1, label`	**relativo a PC**	label se codifica como un número en Ca2: el desplazamiento (en palabras) respecto al PC.

8 Subrutinas y pila

8.1 jal y jr: ir y volver

DEFINICIÓN · JAL (JUMP AND LINK)

`jal subrutina` hace dos cosas: **1)** salta a la subrutina indicada y **2)** guarda la dirección de la **siguiente instrucción** (la de vuelta) en el registro **$31 = $ra**.

`jr $ra` (*jump register*) salta a la dirección guardada en $ra, es decir, **vuelve** justo después del `jal`.

```
        jal  sumar      # $ra ← dirección de "
        # instrucción n   ← aquí se vuelve
        # instrucción x
        ...
sumar:  ...
        ...
        jr   $ra        # PC ← $ra
```

ANALOGÍA

Estás leyendo una novela y una nota al pie te manda al glosario. Antes de ir, pones el **marcapáginas** donde estabas (`jal` guarda $ra). Lees el glosario y vuelves al marcapáginas (`jr $ra`).

Las diapositivas del libro dicen que a $31 "se le llama, por razones históricas, *contador de programa*". Ojo, que no te confunda: el contador de programa es el **PC**; `$31` guarda una **copia** de lo que valía el PC (la dirección de vuelta). Su nombre en MIPS es **$ra**, *return address*.

8.2 El convenio de registros

Para que cualquier subrutina se pueda usar desde cualquier sitio, todos siguen el mismo acuerdo:

Registros	Papel en una llamada
`$a0 – $a3`	**Argumentos**: el que llama pone aquí los datos de entrada
`$v0 – $v1`	**Resultados**: la subrutina deja aquí lo que devuelve
`$t0 – $t9`	Temporales: la subrutina **puede machacarlos** (no se conservan)
`$s0 – $s7`	Guardados: si la subrutina los usa, **debe dejarlos como estaban**
`$ra`	Dirección de retorno
`$sp`	Puntero a la cima de la pila

EJEMPLO · UNA SUBRUTINA QUE SUMA

```c
int suma(int a, int b) {
    return a + b;
}
main() {
    printf("%d", suma(7, 5));
}
```

```
.text
.globl main
main:  li   $a0, 7          # 1er argumento
       li   $a1, 5          # 2º argumento
       jal  suma            # llamada; $ra = siguiente
       move $a0, $v0        # resultado → imprimir
       li   $v0, 1
       syscall              # imprime 12
       li   $v0, 10
       syscall

suma:  add  $v0, $a0, $a1   # resultado en $v0
       jr   $ra             # volver
```

8.3 El problema de las llamadas anidadas

Si A llama a B y **B llama a C**, el `jal C` **sobrescribe $ra**: se pierde la dirección para volver de B a A. Cada `jal` modifica $31, así que en llamadas sucesivas anidadas hay que **guardar las direcciones de retorno**. La solución es usar una **pila en la memoria**.

8.4 La pila

DEFINICIÓN

La **pila** es una zona de memoria que funciona en modo **LIFO** (*Last In, First Out*: lo último que entra es lo primero que sale). El registro **$sp** (= `$29`) apunta a la **cima** (el último dato guardado). En MIPS la pila **crece hacia direcciones más bajas**.

ANALOGÍA

Una **pila de platos**: pones platos encima y siempre coges el de arriba. El $sp es tu dedo señalando el plato de arriba del todo.

Meter (push) $ra en la pila

```
addi $sp, $sp, -4    # hacer hueco (4 bytes)
sw   $ra, 0($sp)     # guardar $ra en la cima
```

Sacar (pop) $ra de la pila

```
lw   $ra, 0($sp)     # recuperar $ra
addi $sp, $sp, 4     # liberar el hueco
```

EJEMPLO DEL LIBRO · A LLAMA A B, Y B LLAMA A C

```
A:   ...
     jal  B               # llama a B; guarda en $31 la vuelta a A
     ...
B:   ...
     addi $29, $29, -4    # hace hueco en la pila
     sw   $31, 0($29)     # guarda la dirección de vuelta a A
     jal  C               # llama a C; $31 = vuelta a B (¡se pisa!)
     lw   $31, 0($29)     # recupera la dirección de vuelta a A
     addi $29, $29, 4     # libera el hueco
     ...
     jr   $31             # vuelve a la siguiente instrucción de A
C:   ...
     jr   $31             # vuelve a la siguiente instrucción de B
```

(La diapositiva usa `add $29,$29,$24` y `sub $29,$29,$24` suponiendo que `$24` vale el tamaño de una palabra y dibujando la pila hacia arriba. Con el convenio de MIPS, la pila crece hacia abajo, así que se resta 4 al guardar y se suma 4 al sacar. La idea es la misma.)

♥ Qué pasa en la pila, paso a paso

Momento	$31 ($ra)	Pila
1. Después de que A llama a B	dirección de vuelta a A	vacía; $sp apunta a la cima
2. Justo antes de que B llame a C	dirección de vuelta a A	**se ha guardado** la vuelta a A; $sp apunta al nuevo tope
3. Después de que B llama a C	dirección de vuelta a **B** (se ha pisado)	sigue guardada la vuelta a A (¡a salvo!)
4. Justo antes de que B vuelva	dirección de vuelta a A (recuperada con lw)	se libera el hueco; $sp vuelve a su sitio

EJEMPLO COMPLETO · SUBRUTINA QUE LLAMA A OTRA

`doble_suma(a, b)` devuelve 2·(a + b) y usa `suma` por dentro:

```
main:        li   $a0, 7
             li   $a1, 5
             jal  doble_suma      # $v0 = 24
             move $a0, $v0
             li   $v0, 1
             syscall
             li   $v0, 10
             syscall

doble_suma:  addi $sp, $sp, -4     # push $ra (vamos a hacer otro jal)
             sw   $ra, 0($sp)
             jal  suma             # $v0 = a + b  (esto pisa $ra)
             add  $v0, $v0, $v0    # $v0 = 2*(a + b)
             lw   $ra, 0($sp)      # pop $ra
             addi $sp, $sp, 4
             jr   $ra              # vuelve a main

suma:        add  $v0, $a0, $a1
             jr   $ra
```

Regla práctica: si una subrutina hace un `jal`, tiene que guardar `$ra` en la pila al principio y recuperarlo antes de su `jr $ra`. Lo mismo con los `$s` que use. Las que no llaman a nadie (*hoja*, como `suma`) no lo necesitan.

9 Ciclo de instrucción del MIPS

Es el ciclo de ejecución del capítulo 1, ahora con los detalles del MIPS. Hay **4 fases**; la 3 y la 4 dependen del tipo de instrucción.

FASE 1 · RECOGIDA (FETCH)

Se lee la instrucción de memoria (usando el PC) y se guarda en el registro de instrucción **IR**. Se actualiza **PC = PC + 4**.

FASE 2 · DECODIFICACIÓN Y LECTURA DE REGISTROS

- Se decodifica la instrucción con el IR.
- Se leen los registros fuente usando los campos rs y rt: en **A** queda el contenido del registro indicado por **IR[25-21]** (rs) y en **B** el del indicado por **IR[20-16]** (rt).
- Por si es un salto condicional, se calcula y guarda la dirección de salto en **TARGET**.

	Aritmética (tipo R)	Memoria (lw / sw)	Salto incondicional	Salto condicional
Fase 3	La ALU ejecuta la operación: **ALUout = A "operación" B**	Se forma la dirección del dato: **ALUout = A + IR[15-0]** (con el signo extendido)	La dirección de salto está en la propia instrucción → se carga en el PC. **Termina.**	La ALU compara; si **ALUout = 0** (iguales), se carga el PC desde TARGET. **Termina.**
Fase 4	Se guarda ALUout en el registro indicado por el campo **rd** del IR	**sw**: se escribe B en la dirección ALUout. **lw**: el dato de la dirección ALUout se escribe en el registro IR[20-16] (rt)	—	—

Fase 1 fetch, PC+4 → **Fase 2** decodificar, leer A y B → **Fase 3** ALU / dirección / salto → **Fase 4** escribir resultado ↻

TRUCO

Los saltos acaban en la fase 3 (solo cambian el PC: no escriben nada). Aritméticas, lw y sw necesitan la fase 4 para dejar el resultado en su sitio.

10 Consejos, fallos típicos y práctica

10.1 Consejos para programar en ensamblador

- **No programes directamente en ensamblador.** Primero haz el diseño (diagrama de flujo, o en C/Java/pseudocódigo) y luego tradúcelo poco a poco.
- **Comenta el código y los datos**: por línea o por grupo de líneas, indicando qué parte del diseño implementa.
- **Prueba con suficientes casos** para comprobar que el programa cumple lo que se pide.

10.2 Fallos típicos

1. **Programa mal planteado**: no hace lo que se pide, o lo hace mal.
2. **Programar directamente en ensamblador**, sin escribir antes el algoritmo en pseudocódigo.
3. **Código ilegible**: sin tabular, sin comentarios o sin relación con el algoritmo inicial.

ERRORES QUE CUESTAN PUNTOS EN EL EXAMEN

- Olvidar el **×4** al calcular la dirección de un elemento de un vector de enteros.
- Usar `lb` con caracteres en vez de `lbu`.
- Confundir `la` (dirección) con `lw` (contenido).
- Olvidar el `j Exit` en un if-else.
- Hacer un `jal` dentro de una subrutina sin guardar antes `$ra`.
- Contar el desplazamiento de un `beq` desde la propia instrucción en vez de desde la siguiente.
- Escribir los campos en binario en el orden del ensamblador (rd, rs, rt) en vez de op, rs, rt, rd.

10.3 Ejercicios para practicar (con solución)

P1 · ARITMÉTICA CON DESPLAZAMIENTO

Con a, b, c en `$t0, $t1, $t2`, calcula **x = (a + b) · 4 − c** en `$t3` sin usar `mul`.

SOLUCIÓN

```
add $t3, $t0, $t1    # a + b
sll $t3, $t3, 2      # ·4
sub $t3, $t3, $t2    # − c
```

P2 · LB FRENTE A LBU

En memoria hay un byte `0x80`. ¿Qué queda en el registro con `lb` y con `lbu`?

SOLUCIÓN

0x80 = 10000000 (empieza por 1). `lb` → `0xFFFFFF80` = **−128**. `lbu` → `0x00000080` = **128**.

P3 · ENDIANNESS

Se ejecuta `sw` de `0x12345678` en la dirección 0x1000. ¿Qué byte hay en cada dirección en una máquina big endian y en una little endian?

SOLUCIÓN

Dirección	Big endian	Little endian
0x1000	12	78
0x1001	34	56
0x1002	56	34
0x1003	78	12

P4 · CODIFICAR SUB

Codifica `sub $t0, $t1, $t2` en binario y hexadecimal (funct de sub = 34).

SOLUCIÓN

$t0 = 8 (rd), $t1 = 9 (rs), $t2 = 10 (rt). op = 0, shamt = 0, funct = 34 = 100010.
`000000 01001 01010 01000 00000 100010` = **0x012A4022** (solo cambia la función respecto a `add`).

P5 · DESPLAZAMIENTO DE UN SALTO

Un `beq` está en la dirección 0x00400020 y la etiqueta destino en 0x00400010. ¿Qué desplazamiento se codifica?

SOLUCIÓN

(0x00400010 - (0x00400020 + 4)) / 4 = (-0x14)/4 = -20/4 = **−5**.

P6 · MODOS DE DIRECCIONAMIENTO

Indica el modo de cada operando: `sw $t1, 8($sp)` · `j fin` · `la $a0, msg` · `lw $t0, ($t3)`.

SOLUCIÓN

`$t1` registro, `8($sp)` relativo a pila · `fin` directo (dirección absoluta en la instrucción) · `$a0` registro, `msg` inmediato · `$t0` registro, `($t3)` indirecto de registro.

P7 · DISEÑO DE FORMATO

Computador de 32 bits, 100 instrucciones y 32 registros. Formato de `OP R1, R2, inm` en una palabra. ¿Qué rango tiene el inmediato si va en complemento a 2?

SOLUCIÓN

100 instrucciones → **7 bits** (2^7 = 128). 32 registros → **5 bits** cada uno. Quedan 32 - 7 - 5 - 5 = **15 bits** para el inmediato → rango en Ca2: **-2^{14} … $2^{14} - 1$ = -16 384 … 16 383**.

P8 · TRADUCIR UN IF

Con x en `$s0` e y en `$s1`: `if (x < y) x = y; else y = 0;`

SOLUCIÓN

```
        bge  $s0, $s1, Else   # condición al revés: si x >= y → else
        move $s0, $s1         # x = y
        j    Exit
Else:   li   $s1, 0           # y = 0
Exit:
```

11 Chuleta de instrucciones

Combina el **cuadro resumen del MIPS-R2000** y la tabla del libro. ★ = instrucción subrayada por tu profe en la tabla del libro. Tipo: **R**, **I**, **J** = formato · **PS** = pseudoinstrucción.

Instrucción	Tipo	Qué hace	Ejemplo
Carga (memoria → registro)			
`lw rt, dir` ★	I	Carga una palabra (32 bits)	`lw $s0, 12($a0)` # $s0 ← Mem[12+$a0]
`lb rt, dir` ★	I	Carga un byte y extiende el signo	`lb $s0, 12($a0)`
`lbu rt, dir` ★	I	Carga un byte sin extender el signo	`lbu $s0, 12($a0)`
`lh / lhu rt, dir`	I	Carga media palabra con / sin extensión de signo	`lh $s0, 12($a0)`
`la reg, dir`	PS	Carga la dirección	`la $s0, VAR`
`lui rt, dato`	I	Dato en los 16 bits altos; bajos a 0	`lui $s0, 12`
`li reg, dato`	PS	Carga un valor inmediato	`li $s0, 12`
Almacenamiento (registro → memoria)			
`sw rt, dir` ★	I	Guarda la palabra	`sw $s0, 12($a0)` # Mem[12+$a0] ← $s0
`sb rt, dir` ★	I	Guarda el byte bajo (LSB)	`sb $s0, 12($a0)`
`sh rt, dir`	I	Guarda los 16 bits bajos	`sh $s0, 12($a0)`
Aritméticas			
`add rd, rs, rt` ★	R	Suma con signo (overflow → excepción)	`add $t0, $a0, $a1`
`addu rd, rs, rt` ★	R	Suma sin signo	`addu $t0, $a0, $a1`
`sub / subu rd, rs, rt` ★	R	Resta con / sin signo	`sub $t0, $a0, $a1`
`addi rt, rs, valor` ★	I	Suma inmediata con signo	`addi $t0, $a0, -24`
`addiu rt, rs, valor` ★	I	Suma inmediata sin signo	`addiu $t0, $a0, 24`
`mult / multu rs, rt` ★	R	Producto: parte alta → HI, parte baja → LO	`mult $s0, $s1`
`div / divu rs, rt` ★	R	Cociente → LO, resto → HI	`div $s0, $s1`
`mul / div / rem rd,rs,rt`	PS	Versiones de 3 operandos	`rem $t0, $t1, $t2`
`mfhi rd` · `mflo rd`	R	rd ← HI · rd ← LO	`mflo $t1`
Lógicas			
`and / or / xor / nor rd,rs,rt` ★	R	Operación bit a bit	`and $t0, $a0, $a1`
`andi / ori / xori rt,rs,inm`	I	Con inmediato (extendido con ceros)	`ori $t0, $a0, 0xA1FF`
`not rd, rs`	PS	Invierte los bits	`not $t0, $t1`
Desplazamiento			
`sll rd, rt, shamt`	R	Lógico a la izquierda	`sll $t0, $t1, 16`
`srl rd, rt, shamt`	R	Lógico a la derecha (entran ceros)	`srl $s0, $t1, 4`
`sra rd, rt, shamt`	R	Aritmético a la derecha (copia el signo)	`sra $s0, $t1, 4`

Comparaciones			
`slt rd, rs, rt` ★	R	rd = 1 si rs < rt	`slt $t0, $a0, $a1`
`slti rt, rs, inm`	I	rt = 1 si rs < inm	`slti $t0, $a0, -15`
`seq / sne / sge / sgt / sle`	PS	==, !=, >=, >, <=	`sge $t0, $a0, $a2`
Saltos condicionales			
`beq rs, rt, etiq` ★	I	Salta si rs == rt	`beq $t0, $t1, DIR`
`bne rs, rt, etiq` ★	I	Salta si rs != rt	`bne $t0, $t1, DIR`
`bgez / bgtz rs, etiq` ★	I	Salta si rs >= 0 / rs > 0	`bgez $t0, SLT`
`blez / bltz rs, etiq` ★	I	Salta si rs <= 0 / rs < 0	`bltz $t1, ETQ`
`bge / bgt / ble / blt`	PS	Comparan dos registros	`blt $t0, $t1, DIR`
`beqz / bnez rs, etiq`	PS	Comparan con 0	`beqz $t2, fin`
Saltos incondicionales			
`j dirección` ★	J	PC ← dirección	`j finbucle`
`jal dirección`	J	Salta y guarda la vuelta en $ra	`jal rutina`
`jr rs` ★	R	PC ← rs	`jr $ra`
`b etiqueta`	PS	Salto relativo al PC	`b bucle`
Especiales y pseudoinstrucciones de movimiento			
`syscall` ★	R	Llamada al sistema (código en $v0)	`li $v0, 10` + `syscall`
`break` ★	R	Ruptura: detiene la ejecución (provoca una excepción; se usa para depurar)	
`move rd, rs`	PS	rd ← rs	`move $a0, $t0`

FORMATOS EN UNA LÍNEA

R: op(6) rs(5) rt(5) rd(5) shamt(5) funct(6) · **I**: op(6) rs(5) rt(5) inm(16) · **J**: op(6) objetivo(26)

12 Erratas corregidas del Tema 3

Esto es lo que estaba mal (o confuso) en el material original y cómo lo he puesto en la guía. Si en clase os lo explicaron de otra manera, haz caso al profe.

Dónde	Pone	Correcto
T3 (I) · Lógicas	`ori $0 $t1 80` ($t0 = $t1 | 80)	`ori $t0, $t1, 80` (no se puede escribir en $0)
T3 (I) · Comparación	`sneq`	El nombre en el simulador y en el cuadro resumen es `sne`
T3 (II) · Arquitectura	"Direcciones de memoria de 2 bits"	Direcciones de **32** bits (2^{32} = 4 GB)
T3 (II) · Hola mundo	`msg_holra`	`msg_hola`
T3 (II) · Direccionamiento	`lbu $a0 0x0FFEE0000`	`lbu $a0, 0xFFEE0000` (sobra un 0)
T3 (II) · Alineamiento	`w1. .word 10`	`w1: .word 10`
T3 (II) · Ejercicio V[5]	`lw $t1, 20`	`li $t1, 20` (cargar el número 20, no leer de memoria)
T3 (II) · Endianness	000110110000000000000000000000000 (33 bits)	0001 1011 0000…0 (32 bits) = 0x1B000000
T3 (III) · Vectores	`la $t0 16`	`li $t0, 16` (16 es un número, no una etiqueta)
Libro · Tabla 12.10	BEQ "Bifurcar sin igual"	BEQ = bifurcar **si** igual
Libro · Tabla 12.10	MULTU "Multiplicar signo"	MULTU = multiplicar **sin** signo
Libro · Salto condicional	`if (1 == j) goto L1;`	`if (i == j) goto L1;`
Libro · While	`mult $9,$19,$10` # $9 = 1*4	`mul $9, $19, $10` # $9 = **i***4 (mult real tiene 2 operandos)
Libro · While	`add $19,419,$20` # i 0 i + j	`add $19, $19, $20` # i = i + j
Libro · Switch	`mult $9, $19, $21` # k*4	`mul $9, $21, $10` ($21 = k, $10 = 4). Además hace falta definir la tabla: `Tabla: .word L0, L1, L2`
Libro · If-else	"saltada se i = j"	"saltada si i = j"
Libro · Vectores	`lw $8, Inicio($16)` con $16 = i	Funciona solo si $16 = i·4; si no, antes `sll $9, $16, 2`
Libro · Subrutinas	`add $29,$29,$24` antes de guardar, `sub` después	Con el convenio MIPS (pila hacia abajo): `addi $sp,$sp,-4` antes, `addi $sp,$sp,4` después
Libro · jal	A $31 "se le llama contador de programa"	$31 es **$ra** (dirección de retorno); el contador de programa es el PC
Varios	"transfenrencia", "longitut", "precición", "esamblador", "Bifucar"	transferencia, longitud, precisión, ensamblador, bifurcar

FUNDAMENTOS DE COMPUTADORES · TEMA 4

Jerarquía de memoria

Tipos de memoria · memoria principal · memoria caché

Tema 4 (I) · Jerarquía de memoria | Tema 4 (II) · Memoria caché

Los dos PDFs repetían mucho (el índice, el ejemplo del bucle, la jerarquía…): aquí cada cosa sale una vez. Todos los ejercicios de las diapositivas están resueltos, también los que venían sin solución. La **memoria virtual** sale en el índice de las diapositivas, pero no viene en estos PDFs.

13 Tipos de memoria y jerarquía

13.1 Dónde vive cada cosa

En el tema 3 viste que la CPU solo trabaja con **registros** y que todo lo demás (programa y datos) está en **memoria**. Ahora miramos la memoria por dentro. En un computador hay tres sitios donde guardar información:

Dónde	Qué guarda	Capacidad	Tiempo de acceso
Banco de registros	Los valores que se usan ahora mismo y la instrucción actual	Muy poca	~1 ns
Memoria principal (RAM)	Programas y datos **en ejecución**	GB	40-100 ns
Disco	Programas y datos que **no** se están ejecutando	Casi ilimitada	milisegundos

La memoria principal está unida al procesador por el **bus**; el disco y los demás periféricos cuelgan de ese bus a través de **módulos de E/S**. Un acceso a memoria principal cuesta **muchos ciclos de reloj** del procesador.

13.2 Tipos de memoria y sus características

♥ Según la tecnología

SEMICONDUCTORAS

Circuitos electrónicos. **RAM, ROM y Flash**.

MAGNÉTICAS

Superficie magnetizada. **Discos duros y cintas**.

ÓPTICAS

Un láser hace pequeñas marcas en la superficie. **CD, DVD, Blu-ray**.

♥ Características que tienes que saber nombrar

Característica	Qué significa
Permanencia	**Volátil**: se borra al quitar la corriente (RAM). **No volátil**: se mantiene (ROM, Flash).
Operaciones	De **lectura y escritura** (RAM) o de **solo lectura** (ROM).
Unidad de almacenamiento	Bits, palabras, bloques…
Modo de acceso	**Secuencial**: hay que pasar por todo lo anterior (cinta). **Aleatorio**: se accede en cualquier orden y se tarda **lo mismo** en llegar a cualquier posición (RAM).
Tiempo de acceso	Tiempo desde que se presenta la dirección hasta que se obtiene el dato.
Ancho de banda	(Velocidad de transferencia) cantidad de datos por unidad de tiempo, p. ej. GB/s.
Capacidad y **coste**	Cuánto cabe y cuánto cuesta cada unidad almacenada (€/MB).

ANALOGÍA

Una cinta es como una canción en un casete: para llegar al minuto 3 tienes que pasar por los minutos 1 y 2 (secuencial). Una RAM es como una playlist de Spotify: saltas directamente a la canción que quieras y tardas lo mismo sea cual sea (aleatorio).

13.3 Unidades de tamaño

Unidad	Equivale a	Potencia
1 byte	8 bits	—
1 KB	1024 bytes	2^{10} B
1 MB	1024 KB	2^{20} B
1 GB	1024 MB	2^{30} B
1 TB	1024 GB	2^{40} B
1 PB · EB · ZB · YB	×1024 cada vez	$2^{50} \cdot 2^{60} \cdot 2^{70} \cdot 2^{80}$

¡OJO! DOS TRAMPAS

- **b minúscula = bit, B mayúscula = byte**: 1 Kb ≠ 1 KB.
- En **comunicaciones** y en los **discos duros** se suelen usar potencias de **10**: 1 Kb = 1000 bits, 1 GB = 10^9 bytes. Por eso un disco "de 1 TB" aparece en Windows con unos 931 GB.

TRUCO

Para pasar de potencia de 2 a nombre: 2^{10} = K, 2^{20} = M, 2^{30} = G. Así $2^{26} = 2^6 \cdot 2^{20}$ = **64 M**. Lo usarás muchísimo en los ejercicios de caché.

13.4 El problema: la memoria no puede seguir al procesador

Entre 1980 y 2000 los procesadores mejoraron un **60 % al año**; las memorias DRAM, solo un **7 %**. Cada año la distancia es mayor: el procesador pasa la mayor parte del tiempo **esperando a la memoria**.

CÓMO CONTAR ACCESOS A MEMORIA

Cada instrucción que se ejecuta supone **1 acceso** para traerla (fetch, capítulo 9). Las de carga y almacenamiento (`lw` , `sw` …) suponen **1 acceso más** para el dato. Las que solo usan registros no tocan la memoria otra vez.

EJEMPLO 1 · SUMA DE 0 A 999

```
int i;
int s = 0;
for (i = 0; i < 1000; i++)
    s = s + i;
```

```
        li   $t0, 0       # s
        li   $t1, 0       # i
        li   $t2, 1000
bucle:  bge  $t1, $t2, fin
        add  $t0, $t0, $t1
        addi $t1, $t1, 1
        b    bucle
fin:    li   $t1, 0
```

- 3 `li` del principio → **3**
- 4 instrucciones por vuelta × 1000 vueltas → **4000**
- El último `bge`, el que sale del bucle (cuando i = 1000) → **1**
- El `li` de `fin` → **1**

Total = 3 + 4·1000 + 1 + 1 = 4005 accesos. Con una memoria de 60 ns: 4005 × 60 = **240 300 ns**. Un procesador típico pasaría más del **98 %** de ese tiempo esperando.

EJEMPLO 2 · PONER UN VECTOR A CERO (CON SW)

```
int v[1000];   // global
int i;
for (i = 0; i < 1000; i++)
    v[i] = 0;
```

```
        .data
v:      .space 4000
        .text
        li   $t0, 0       # i
        li   $t1, 0       # despl. en v
        li   $t2, 1000
bucle:  bge  $t0, $t2, fin
        sw   $0, v($t1)
        addi $t0, $t0, 1
        addi $t1, $t1, 4
        b    bucle
fin:
```

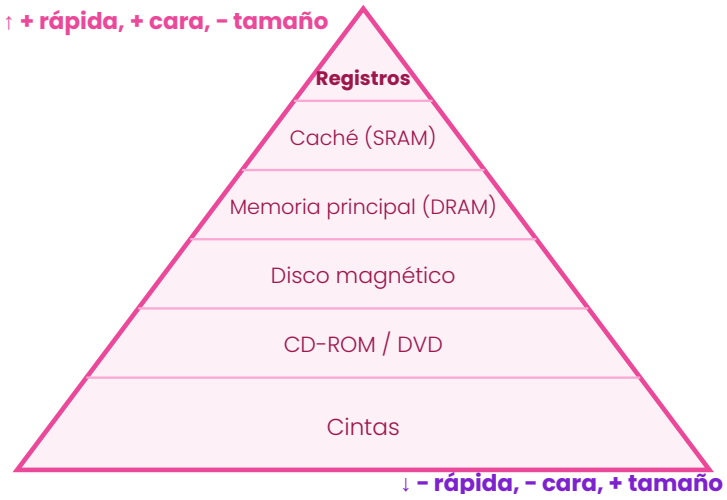
3 + **5**·1000 + 1 (bge final) + **1000** (el dato de cada `sw`) = **6004 accesos**. Aquí no hay instrucción en `fin`, así que no se suma.

TRUCO

Haz siempre la misma cuenta: **antes del bucle + (instrucciones del bucle × vueltas) + la comparación que sale + lo de después + 1 por cada lw/sw ejecutado**.

13.5 La jerarquía de memoria

La memoria ideal sería **muy rápida**, **muy grande** y **muy barata**. Pero esos objetivos son **incompatibles**: cuanto más rápida es una memoria, más cara y más pequeña. La solución es usar **varios tipos** de memoria y ordenarlos por velocidad: eso es la jerarquía de memoria.



Arriba: rápida, cara y pequeña. Abajo: lenta, barata y enorme.

ANALOGÍA · LOS LIBROS

Necesitas un libro para estudiar:

- Si está **en tu mesa** (registro, ~1 ns), lo coges al instante.
- Si está en la **biblioteca de la facultad** (caché SRAM, ~2-5 ns), tardas un poco.
- Si está en **una biblioteca de Florida** (DRAM, ~70-100 ns), es un viaje.

Por eso, cuando vas a la biblioteca no traes una sola hoja: traes el libro entero y lo dejas en la mesa por si lo vuelves a usar.

♥ Cómo se usa

- En cada nivel se guarda **solo lo que se necesita en ese momento**.
- Si un dato no está en un nivel, se **copia** desde el nivel de abajo la **porción** que hace falta (por ejemplo, al ejecutar un programa se carga del disco a la RAM).
- Cuando ya no se necesita, esa copia se borra.
- Funciona porque los programas cumplen el **principio de proximidad** (localidad de referencias), que verás en el capítulo 15.

Tecnología	Bytes por acceso	Latencia	Coste por MB	Energía por acceso
Caché dentro del chip	10	100 picosegundos	1-100 $	1 nJ
Caché fuera del chip	100	nanosegundos	1-10 $	10-100 nJ
DRAM	1000 (leídos por dentro)	10-100 ns	0,1 $	1-100 nJ
Disco	1000	milisegundos	0,001 $	100-1000 mJ

♥ ¿Por qué es tan importante en los multicore?

El ancho de banda que necesita el procesador crece con el número de núcleos. Ejemplo: un Intel Core i7 con **4 núcleos a 3,2 GHz** hace 2 accesos a memoria por núcleo y por ciclo:

- Datos: $4 \times 3{,}2 \cdot 10^9 \times 2$ = **25,6 mil millones** de accesos de 64 bits (8 B) por segundo → 204,8 GB/s
- Instrucciones: **12,8 mil millones** de accesos de 128 bits (16 B) por segundo → 204,8 GB/s
- Total: **409,6 GB/s**. Una DRAM ofrece unos 25 GB/s, solo el **6 %** → hacen falta **memorias multipuerto** y **varios niveles de caché**.

RESUMEN DEL CAPÍTULO 13

Registros (ns) → RAM (decenas de ns) → disco (ms). Contar accesos: 1 por instrucción ejecutada + 1 por cada lw/sw. Rápida, grande y barata a la vez es imposible → se combinan memorias en una jerarquía, y en cada nivel solo se guarda una copia de lo que se usa ahora.

14 La memoria principal

14.1 ROM y RAM

ROM · SOLO LECTURA

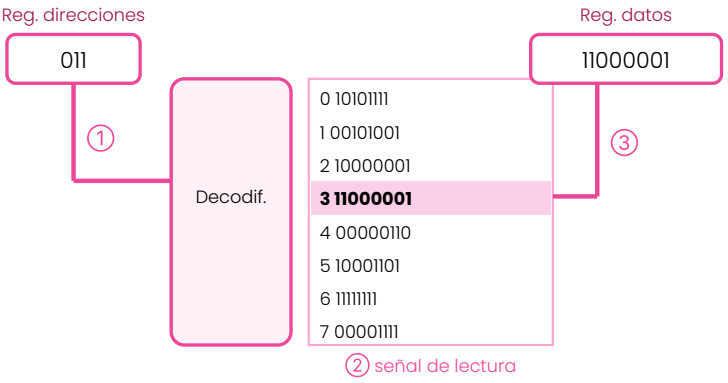
- **No necesita** alimentación
- **Persistente** (no volátil)
- Ejemplo: la **BIOS**

RAM · LECTURA Y ESCRITURA

- **Necesita** alimentación
- **No persistente** (volátil)
- **Más rápida** que la ROM
- Ejemplo: la **memoria principal**

14.2 Cómo se lee una memoria

Por dentro, una memoria es una **matriz**: cada fila es una posición (aquí, un byte) y tiene su número, que es su dirección.



1. La dirección (**011** = 3) llega por el **bus de direcciones** y el **decodificador** activa la fila 3.
2. Se activa la señal de **lectura**.
3. El contenido de la fila 3 (**11000001**) sale por el **bus de datos** al registro de datos.

Con 3 bits de dirección hay 2^3 = 8 posiciones. En general, **n bits de dirección → 2^n posiciones**.

14.3 DRAM y SRAM

	DRAM (dinámica)	SRAM (estática)
Cómo guarda un bit	Como **carga en un condensador**	Como un **interruptor** encendido o apagado
¿Se descarga?	Sí → necesita **refresco** periódico	No → **no necesita refresco**
Ventajas	Más simple, **más capacidad**, **más barata**	**Más rápida**, sin circuitería de refresco
Inconvenientes	Necesita circuitería de refresco, **más lenta**. El refresco gasta un 2-3 % de los ciclos	Construcción compleja, **menos capacidad**, **más cara**
Se usa en	**Memoria principal** (los módulos de RAM de la placa)	**Memorias caché**

ANALOGÍA

La DRAM es un vaso de agua con un agujerito: si no lo rellenas cada poco, se vacía (por eso hay que refrescarla). La SRAM es un interruptor de la luz: se queda como lo dejes mientras haya corriente.

♥ El refresco

La carga del condensador se pierde con el **tiempo** y la **temperatura**, así que cada bit se tiene que refrescar cada **pocos milisegundos**. Una **lectura refresca toda la fila** que contiene esa dirección, así que la DRAM usa **ciclos de refresco** que van leyendo fila a fila.

14.4 Tipos de DRAM

Para leer en una DRAM se da primero la **fila** y luego la **columna**. Los tipos se diferencian en cómo encadenan varias lecturas seguidas:

Tipo	Qué mejora
FPM (*Fast Page Mode*)	La fila se da **una vez** y después se leen varias columnas de esa fila.
EDO (*Extended Data Output*)	Mientras sale un dato ya se puede dar la siguiente columna: los accesos se **solapan**.
SDRAM (*Synchronous DRAM*)	Va **sincronizada con el reloj** del sistema y entrega varios datos seguidos (ráfaga).
DDR, DDR2, DDR3, DDR4	SDRAM de *Double Data Rate*: transfiere **dos datos por ciclo** de reloj (en el flanco de subida y en el de bajada).

EJEMPLO · LEER EL NOMBRE DE UN MÓDULO DDR

Un módulo **DDR4-3200** funciona con un reloj de **1600 MHz**. Como hace 2 transferencias por ciclo: 1600 × 2 = **3200 millones de transferencias/s** (el número del nombre). Cada transferencia es de 64 bits = 8 bytes: 3200 × 8 = **25 600 MB/s**, de ahí su otro nombre, **PC25600**.

Estándar	Reloj (MHz)	Millones de transf./s	Nombre DRAM	MB/s	Nombre DIMM
DDR	133-200	266-400	DDR266 … DDR400	2128-3200	PC2100 … PC3200
DDR2	266-400	533-800	DDR2-533 … DDR2-800	4264-6400	PC4300 … PC6400
DDR3	533-800	1066-1600	DDR3-1066 … DDR3-1600	8528-12 800	PC8500 … PC12800
DDR4	1066-1600	2133-3200	DDR4-3200	17 056-25 600	PC25600

♥ Cada vez más grandes, no mucho más rápidas

Según los datos de Patterson y Hennessy, de **1980** a **2012** un chip DRAM pasó de **64 Kbit a 8 Gbit**, es decir, unas **131 000 veces más capacidad**. En cambio, su tiempo de ciclo solo bajó de **250 ns a 31 ns**, unas **8 veces**. Esta es la brecha del capítulo 13 vista con números.

14.5 Tipos de ROM

Tipo	Cómo se escribe / borra
ROM de máscara	Los datos se graban **al fabricarla**. No se pueden cambiar.
PROM (ROM programable)	El usuario la programa **una sola vez**.
EPROM (PROM borrable)	Se puede borrar y volver a programar. La **UV EPROM** se borra con **luz ultravioleta**.
EEPROM	Se borra **eléctricamente**, sin sacarla del circuito. La memoria **Flash** es de esta familia.

14.6 La memoria principal premia los accesos seguidos

Leer **5 posiciones consecutivas** es mucho más rápido que leer 5 posiciones sueltas. Una vez localizada la primera, las siguientes de la misma fila tienen un coste menor (es justo lo que aprovechan FPM, EDO y las ráfagas de SDRAM). Por eso a la memoria principal le conviene que le pidan **bloques** de palabras y no palabras sueltas: esa es la idea de la caché.

RESUMEN DEL CAPÍTULO 14

ROM: no volátil, solo lectura (BIOS). RAM: volátil, lectura/escritura. DRAM = condensador, refresco, barata y grande → memoria principal. SRAM = interruptor, sin refresco, rápida y cara → caché. DDR = 2 transferencias por ciclo. Los accesos a posiciones consecutivas salen más baratos → conviene traer bloques.

15 La memoria caché: funcionamiento y rendimiento

15.1 El principio de proximidad

DEFINICIÓN

Principio de proximidad (o de **localidad de referencias**): mientras se ejecuta un programa, las direcciones a las que accede tienden a estar agrupadas.

- **Proximidad temporal**: si accedes a una dirección, es probable que **vuelvas a acceder a ella** pronto. Ejemplo: las instrucciones de un **bucle**.
- **Proximidad espacial**: si accedes a una dirección, es probable que accedas pronto a las **de al lado**. Ejemplo: una **secuencia de instrucciones** o recorrer un **vector**.

```
        .data
vector: .space 4*1024
        .text
main:   li   $t0, 0
        la   $t1, vector
b2:     bge  $t0, 1024, finb2
        mul  $t2, $t0, 4
        add  $t2, $t1, $t2
        sw   $t0, ($t2)
        addi $t0, $t0, 1
        b    b2
finb2:  jr   $ra
```

Temporal: las 6 instrucciones de `b2` se repiten 1024 veces.

Espacial: las instrucciones están seguidas en memoria, y el `sw` escribe en `vector[0]`, `vector[1]`, `vector[2]`… que están de 4 en 4 bytes.

ANALOGÍA

Temporal: la canción que te gusta la vuelves a poner en un rato. Espacial: si abres el capítulo 3 de un libro, lo siguiente que leerás será el 4, no el 27.

15.2 Qué es la caché

DEFINICIÓN

Memoria caché: memoria **pequeña y rápida**, de tecnología **SRAM** e integrada en el mismo procesador, situada **entre el procesador y la memoria principal**. Guarda una **copia** de partes de la memoria principal.



La clave: cuando hay que ir a memoria principal no se trae solo la palabra pedida, sino **el bloque entero** que la contiene (la palabra y sus vecinas). Así se aprovecha la proximidad espacial, y como el bloque se queda en la caché, también la temporal.

15.3 Acierto y fallo

1. El procesador pide el contenido de una dirección (por ejemplo con `lw $t0, ($a0)`, o simplemente al ir a buscar la siguiente instrucción).
2. La caché comprueba si tiene esos datos.
   - **ACIERTO** (están): se los da al procesador **rápidamente**.
   - **FALLO** (no están): la caché trae de memoria principal el **bloque** que contiene esa dirección y **después** entrega el dato al procesador.

DEFINICIÓN

Tasa de aciertos (h): probabilidad de que un dato accedido esté en la caché = aciertos / accesos totales. La tasa de fallos es 1 - h.

♥ Ejemplo completo: el bucle de la suma, con y sin caché

Es el ejemplo 1 del capítulo 13 (sin el `li` de `fin`, así que son **3 + 4·1000 + 1 = 4004 accesos**). Datos:

- Acceso a caché: **2 ns**
- Acceso a memoria principal: **120 ns**
- Bloque: **4 palabras** (16 bytes)
- Traer un bloque de MP a caché: **200 ns**

Dir.	Instrucción	Bloque
1000	`li $t0, 0`	A
1004	`li $t1, 0`	
1008	`li $t2, 1000`	
1012	`bge $t1,$t2,fin`	
1016	`add $t0,$t0,$t1`	B
1020	`addi $t1,$t1,1`	
1024	`b bucle`	

Lo que pasa: se pide 1000 → **fallo**, se trae el bloque A entero (1000-1012). Las peticiones de 1004, 1008 y 1012 son **aciertos**. Se pide 1016 → **fallo**, se trae el bloque B. A partir de ahí **todo** está en la caché: las 1000 vueltas son aciertos.

SIN CACHÉ

4004 × 120 ns = **480 480 ns** (≈ 0,48 ms)

CON CACHÉ

2 fallos × 200 ns = 400 ns
4004 accesos × 2 ns = 8008 ns
Total = 8408 ns

Tasa de aciertos = 4002 / 4004 = **99,95 %**. ¡Es unas **57 veces** más rápido!

EJERCICIO · TASA DE ACIERTOS SUMANDO UN VECTOR

Con los mismos tiempos y bloques de 4 palabras, calcula la tasa de aciertos de este código (el vector empieza al principio de un bloque y el código también):

```
int v[1000];  // global
int i, s;
for (i = 0; i < 1000; i++)
    s = s + v[i];
```

```
        li   $t0, 0       # i
        li   $t1, 0       # despl. en v
        li   $t2, 1000
        li   $t3, 0       # s
bucle:  bge  $t0, $t2, fin
        lw   $t4, v($t1)
        add  $t3, $t3, $t4
        addi $t0, $t0, 1
        addi $t1, $t1, 4
        b    bucle
fin:
```

SOLUCIÓN

Accesos: instrucciones 4 + 6·1000 + 1 = 6005, más 1000 lecturas de datos (`lw`) → **7005 accesos**.

Fallos de instrucciones: el código son 10 instrucciones = 40 bytes → ocupa **3 bloques** de 16 bytes → 3 fallos (la primera vez que se entra en cada bloque).

Fallos de datos: v ocupa 4000 bytes = 250 bloques. Se recorre en orden, así que hay 1 fallo al empezar cada bloque (v[0], v[4], v[8]…) y los otros 3 accesos del bloque son aciertos → **250 fallos**.

Tasa de aciertos = (7005 - 253) / 7005 = 6752 / 7005 ≈ **96,4 %**.
Tiempo: 7005 × 2 + 253 × 200 = **64 610 ns**, frente a 7005 × 120 = 840 600 ns sin caché.

Fíjate: los datos de un vector que se lee una sola vez solo aprovechan la proximidad **espacial** (1 fallo de cada 4). Las instrucciones del bucle aprovechan además la **temporal**.

15.4 ¿Por qué funciona?

- El tiempo de acceso a la caché es **mucho menor** que el de la memoria principal. Por ejemplo, DRAM: 20-50 ns; SRAM: menos de 1-2,5 ns.
- A la memoria principal se accede **por bloques**, que es lo que mejor se le da (capítulo 14.6).
- Gracias a la **proximidad temporal y espacial**, casi siempre lo que se pide ya está en la caché.

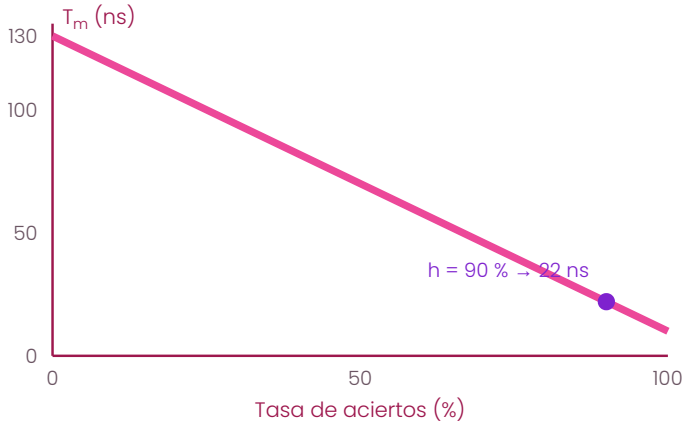
15.5 Tiempo medio de acceso

FÓRMULA CLAVE

$$T_m = h \cdot T_a + (1 - h) \cdot (T_a + T_f) = T_a + (1 - h) \cdot T_f$$

- **T_a**: tiempo de acceso a la caché.
- **T_f**: tiempo de tratar un fallo (reemplazar un bloque y traerlo de memoria principal a la caché). También lo llaman **penalización por fallo**.
- **h**: tasa de aciertos (en tanto por uno: 95 % → 0,95).

De dónde sale: con probabilidad **h** hay acierto y tardas T_a. Con probabilidad **1 − h** hay fallo: tardas T_f en traer el bloque y luego T_a en dárselo al procesador. Si desarrollas: $h{\cdot}T_a + T_a - h{\cdot}T_a + (1 - h){\cdot}T_f = T_a + (1 - h){\cdot}T_f$.



EJEMPLO · T_A = 10 NS, T_F = 120 NS

h	$T_m = 10 + (1-h){\cdot}120$
0 %	130 ns
50 %	70 ns
90 %	22 ns
99 %	11,2 ns
100 %	10 ns

Es una recta: T_m baja de forma lineal al subir h. Para acercarse a la velocidad de la caché hacen falta tasas **muy altas**.

EJERCICIO · TASA DE ACIERTOS NECESARIA

Tiempo de acceso a caché: 4 ns. Tiempo de acceso a un bloque de MP: 120 ns. a) Si h = 90 %, ¿cuál es el tiempo medio de acceso? b) ¿Qué tasa de aciertos hace falta para que sea menor de 10 ns? ¿Y de 5 ns?

SOLUCIÓN

a) $T_m = 0{,}9 \cdot 4 + 0{,}1 \cdot (4 + 120) = 3{,}6 + 12{,}4 =$ **16 ns**.

b) $4 + (1 - h) \cdot 120 < 10 \rightarrow (1 - h) < 6/120 = 0{,}05 \rightarrow$ **h > 0,95** (95 %).
$4 + (1 - h) \cdot 120 < 5 \rightarrow (1 - h) < 1/120 \rightarrow h > 1 - 0{,}00833 \rightarrow$ **h > 0,9917** (99,17 %).

TRUCO PARA DESPEJAR H

Usa siempre la forma corta: **$1 - h = (T_m - T_a) / T_f$**. Te ahorras los errores de signo.

15.6 Niveles de caché

Lo habitual es tener **tres niveles**:

Nivel	Dónde está	Tamaño y velocidad
L1	Interna, la más cercana al procesador. Suele estar **separada en instrucciones y datos**.	Pequeña (8-128 KB), la más rápida
L2	Interna, entre L1 y L3 (o entre L1 y memoria principal).	Mediana (256 KB-4 MB), más lenta que L1
L3	Normalmente el último nivel antes de la memoria principal. Puede ser interna o externa.	Más grande y más lenta que L2

En los procesadores multinúcleo (las diapositivas muestran un AMD de 4 núcleos y un Intel Core i7), cada núcleo suele tener su L1 y su L2, y la L3 la comparten todos.

RESUMEN DEL CAPÍTULO 15

Proximidad temporal (bucles) y espacial (secuencias, vectores). Caché = SRAM entre CPU y MP, guarda copias por bloques. Acierto → rápido; fallo → traer bloque y luego servir. $\mathbf{T_m = T_a + (1 - h)\cdot T_f}$. Niveles L1 (más pequeña y rápida), L2, L3.

16 Organización de la caché

Al diseñar una caché hay que decidir cuatro cosas: su **tamaño** (total y de cada línea), la **función de correspondencia** (dónde va cada bloque), el **algoritmo de sustitución** (a quién se echa cuando no hay sitio) y la **política de escritura**. Es normal que L1, L2… tengan diseños distintos. El tamaño se decide con estudios sobre programas muy usados. Las otras tres decisiones se explican en este capítulo.

16.1 Bloques y líneas

- La memoria principal se divide (lógicamente) en **bloques** de igual tamaño: 1 bloque = k palabras.
- La caché se divide en **líneas**. Una línea es el hueco donde cabe un bloque: **tamaño de línea = tamaño de bloque**.
- La caché es mucho más pequeña, así que **caben muy pocos bloques**. Por eso, junto a cada línea se guarda una **etiqueta** que dice qué bloque de MP hay ahí.

LAS TRES CUENTAS BÁSICAS

Nº de bloques de MP = tamaño MP / tamaño de bloque
Nº de líneas de la caché = tamaño caché / tamaño de línea
Nº de bloque de una dirección = dirección / tamaño de bloque (división entera)

EJERCICIO

¿Cuántos bloques de 4 palabras hay en una memoria de 1 GB?

SOLUCIÓN

4 palabras = 16 B = 2^4 B. 2^{30} / 2^4 = 2^{26} bloques = **64 M bloques** (unos 64 millones).

EJERCICIO

¿Cuántos bloques de 4 palabras caben en una caché de 32 KB?

SOLUCIÓN

2^{15} / 2^4 = 2^{11} = **2048 líneas**.

EJEMPLO · DIRECCIÓN → PALABRA

Computador de 32 bits, memoria direccionada por bytes y palabras de 4 bytes. La palabra que empieza en la dirección `0x00000040` (= 64) es la palabra número 64 / 4 = **16**. Las direcciones de palabra son siempre múltiplos de 4 (acaban en 00 en binario).

16.2 Cómo se parte una dirección

Para buscar una dirección en la caché, sus bits se reparten en campos. Los bits **de la derecha** siempre dicen **qué byte dentro de la línea** (con líneas de 2^b bytes, son b bits). El resto es el **número de bloque**, y cómo se usa depende de la función de correspondencia:

etiqueta	línea o conjunto (según el tipo)	byte

¿Por qué hace falta la etiqueta? Porque **muchos bloques distintos van a parar a la misma línea**. Si en una línea hay datos, la caché tiene que saber de qué bloque son: en la caché de ejemplo de abajo, la dirección 0 y la dirección 32 van a la misma línea (la 0). Si se pide la 32 y en esa línea está el bloque de la dirección 0, es un fallo aunque la línea tenga datos. La etiqueta (los bits altos de la dirección) se guarda junto a la línea para distinguirlos.

♥ La caché de ejemplo de las diapositivas

Caché de **32 bytes**, líneas de **2 palabras** (8 bytes) → 32 / 8 = **4 líneas**. Direcciones de **7 bits** (memoria de 128 bytes = 16 bloques de 8 bytes). Con ella veremos los tres tipos de correspondencia usando la dirección **0100100** (= 36), que está en el bloque 36 / 8 = **4**.

16.3 Correspondencia directa

DEFINICIÓN

Cada bloque de MP tiene **una sola línea posible**, siempre la misma: el bloque **K** va a la línea **K mod nº de líneas**.

etiqueta el resto	línea $\log_2$(nº líneas) bits	byte $\log_2$(tamaño línea) bits

EJEMPLO · DIRECCIÓN 0100100 EN LA CACHÉ DE 4 LÍNEAS

01 etiqueta	00 línea	100 byte

Byte: 8 bytes por línea → 3 bits (**100** = byte 4). Línea: 4 líneas → 2 bits (**00** = línea 0). Etiqueta: lo que sobra (**01**). Comprobación: bloque 4 mod 4 = 0 ✓. Los bloques 0, 4, 8 y 12 compiten todos por la línea 0, y la etiqueta (00, 01, 10, 11) dice cuál de ellos está.

Cómo busca la caché: (1) con el campo **línea** va directa a una línea; (2) compara la **etiqueta** guardada con la de la dirección; (3) si coinciden → **acierto** y el campo **byte** elige el dato; (4) si no → **fallo**, se va a MP.

EJEMPLO · DIRECCIONES DE 32 BITS, CACHÉ DE 64 KB, LÍNEAS DE 8 BYTES

MP: 2^{32} / 2^3 = 2^{29} bloques. Caché: 2^{16} / 2^3 = **2^{13} líneas**. Byte: 3 bits. Línea: 13 bits. Etiqueta: 32 - 13 - 3 = 16 bits.

etiqueta · 16 bits	línea · 13 bits	byte · 3

VENTAJAS

Sencilla y barata: solo hay que comparar una etiqueta.

INCONVENIENTE

Puede tener **muchos fallos** según el patrón de accesos: si un programa usa a la vez dos bloques de la misma línea, se echan el uno al otro continuamente.

EJERCICIO · CORRESPONDENCIA DIRECTA

Computador de 32 bits, caché de 64 KB, bloques de 32 bytes, correspondencia directa. a) ¿En qué línea va la palabra de la dirección `0x0000408A`? b) ¿Cómo se obtiene rápido? c) ¿Y la de `0x1000408A`? d) ¿Cómo sabe la caché cuál de las dos tiene guardada?

SOLUCIÓN

Líneas = $2^{16} / 2^5 = 2^{11} = 2048$. Campos: byte **5** bits, línea **11** bits, etiqueta 32 - 11 - 5 = **16** bits.

a) `0x0000408A` = 0000 0000 0000 0000 **0100 0000 100**0 1010. Los 5 bits de la derecha (01010) son el byte (10). Los 11 siguientes, **01000000100** = **línea 516**. (Comprobación: 0x408A = 16 522; 16 522 / 32 = 516 → bloque 516; 516 mod 2048 = 516 ✓.)

b) Sin dividir: se pasa a binario y se **cogen los bits del campo línea** (bits 15 a 5). Hacer "mod 2^n" es quedarse con los n bits de abajo.

c) `0x1000408A` solo cambia en los bits altos, que son la etiqueta: va a la **misma línea 516**.

d) Por la **etiqueta** guardada en la línea: `0x0000` para la primera dirección y `0x1000` para la segunda.

16.4 Correspondencia asociativa

DEFINICIÓN

Cada bloque de MP puede ir a **cualquier línea** de la caché.

etiqueta = número de bloque	byte
el resto	

Como no hay campo de línea, la etiqueta es el número de bloque entero. En el ejemplo de 7 bits, **0100100** → etiqueta **0100** (bloque 4) y byte **100**. Con 32 bits y líneas de 8 bytes: etiqueta de **29 bits** y byte de 3.

Cómo busca: compara la etiqueta de la dirección con las etiquetas de **todas** las líneas a la vez. Si alguna coincide, el bloque está ahí.

VENTAJA

No depende del patrón de accesos: un bloque solo tiene que salir si la caché está **llena**.

INCONVENIENTES

Búsqueda **costosa** (un comparador por línea) y etiquetas más grandes → caché más grande.

16.5 Correspondencia asociativa por conjuntos

DEFINICIÓN

Las líneas se agrupan en **conjuntos** de K líneas (caché de **K vías**). El bloque **B** va siempre al **conjunto B mod nº de conjuntos**, pero dentro de él puede ir a **cualquiera** de sus K líneas.

Nº de conjuntos = nº de líneas / K

etiqueta el resto	conjunto $\log_2$(nº conjuntos) bits	byte

EJEMPLO · 0100100 EN LA CACHÉ DE 4 LÍNEAS, 2 VÍAS

4 líneas / 2 vías = **2 conjuntos** → 1 bit de conjunto.

010 etiqueta	0 conjunto	100 byte

Bloque 4 mod 2 = **conjunto 0** ✓. Si las dos líneas del conjunto 0 ya están ocupadas (por ejemplo, con los bloques 0 y 2), hay que **echar** uno de ellos para meter el bloque 4.

EJEMPLO · 32 BITS, CACHÉ DE 64 KB DE 4 VÍAS, LÍNEAS DE 8 BYTES

Líneas: 2^{13}. Conjuntos: 2^{13} / 4 = $\mathbf{2^{11}}$. Byte 3 bits, conjunto 11 bits, etiqueta 32 − 11 − 3 = 18 bits.

etiqueta · 18 bits	conjunto · 11 bits	byte · 3

Cómo busca: con el campo **conjunto** elige el conjunto y compara la etiqueta solo con las **K líneas** de ese conjunto.

Es un **compromiso** entre flexibilidad y coste: más flexible que la directa y menos costosa que la asociativa. Tiene lo mejor de las dos y la búsqueda sale menos costosa.

♥ Las tres, de un vistazo

	Directa	Asociativa	Por conjuntos (K vías)
¿Dónde va el bloque B?	Línea B mod nº líneas	Cualquier línea	Cualquier línea del conjunto B mod nº conjuntos
Campos	etiqueta · línea · byte	etiqueta · byte	etiqueta · conjunto · byte
Comparaciones	1	Todas las líneas	K
Pros / contras	Barata · muchos fallos por conflicto	Pocos fallos · cara	Equilibrio

TRUCO · SON LA MISMA IDEA

La directa es una caché por conjuntos de **1 vía** (cada conjunto es una línea). La asociativa es una caché por conjuntos con **un único conjunto** que tiene todas las líneas. Si te aprendes la de conjuntos, te sabes las tres.

ANALOGÍA · EL APARCAMIENTO

Directa: cada coche tiene su plaza fija según su matrícula; si dos coches tienen la misma plaza, el último echa al otro aunque haya plazas libres. **Asociativa**: aparcas donde quieras, pero para encontrar tu coche tienes que mirar todas las plazas. **Por conjuntos**: tu matrícula te dice la planta, y en esa planta aparcas donde quieras.

16.6 Algoritmos de sustitución

Cuando llega un bloque y no hay hueco, hay que elegir qué línea se libera:

- **Directa**: no hay elección; sale el que estuviera en su línea.
- **Asociativa**: se elige una línea de toda la caché.
- **Por conjuntos**: se elige una línea del conjunto que toca.

Algoritmo	Echa a la línea…
FIFO (*First In, First Out*)	que lleva **más tiempo en la caché**
LRU (*Least Recently Used*)	que lleva **más tiempo sin usarse**
LFU (*Least Frequently Used*)	que se ha usado **menos veces**

EJEMPLO · FIFO FRENTE A LRU

Un conjunto de 2 líneas. Accesos a los bloques: **A, B, A, C**. Al llegar C no hay sitio:

- **FIFO** echa a **A**, que fue el primero en entrar… aunque se acaba de usar.
- **LRU** echa a **B**, que es el que lleva más tiempo sin usarse. Normalmente acierta más, gracias a la proximidad temporal.

16.7 Políticas de escritura

Si el procesador modifica un dato que está en la caché, la memoria principal se queda con el valor viejo. En algún momento hay que actualizarla:

	Escritura inmediata	Escritura diferida (post-escritura)
Cómo funciona	Se escribe **a la vez** en la caché y en la MP	Se escribe **solo en la caché** y se marca la línea con un bit que dice que no está volcada a MP. Se escribe en MP **al sustituir el bloque** (o cuando hay poco tráfico con MP)
Ventaja	**Coherencia**: MP siempre está al día	**Velocidad**
Inconvenientes	**Mucho tráfico** con MP y **escrituras lentas**	Problemas de **coherencia**: MP puede tener datos viejos (**inconsistencia**)

¡OJO! CUÁNDO DA PROBLEMAS LA COHERENCIA

- **CPU multinúcleo con una caché por núcleo**: lo que un núcleo escribe en su caché no lo ven los demás. Si dos núcleos escriben la misma palabra, ¿cuál es el resultado final?
- **Módulo de E/S con acceso directo a memoria (DMA)**: escribe o lee en MP sin pasar por la caché, así que lo que actualice puede no ser coherente con la caché.

Dato útil: solo alrededor del **15 %** de los accesos a memoria son escrituras.

LA MISMA IDEA, EN TODAS PARTES

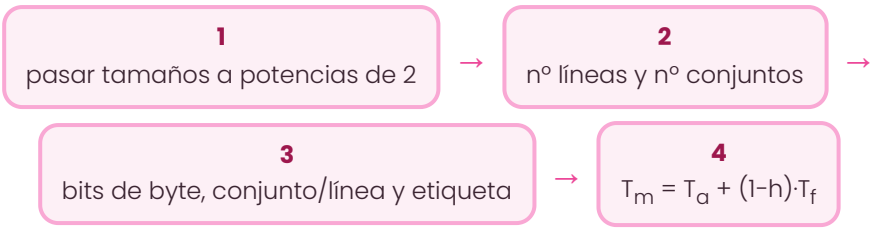
La caché no es solo cosa del procesador. El sistema operativo guarda en RAM trozos del disco (**buffer cache**), tu navegador guarda páginas ya visitadas, un **proxy web** guarda páginas para muchos navegadores y los servidores DNS guardan direcciones ya consultadas. En todos los casos es lo mismo: una copia pequeña y cercana de algo grande y lejano.

RESUMEN DEL CAPÍTULO 16

Línea = hueco de un bloque; la etiqueta dice qué bloque hay. Directa: línea = B mod nº líneas (etiqueta|línea|byte). Asociativa: cualquier línea (etiqueta|byte). Por conjuntos: conjunto = B mod nº conjuntos (etiqueta|conjunto|byte). Sustitución: FIFO, LRU, LFU. Escritura inmediata (coherente, lenta) o diferida (rápida, problemas de coherencia).

17 Ejercicios de examen de caché

Son los ejercicios largos del final de las diapositivas, resueltos paso a paso. Casi todos siguen la misma receta:



E1 · CACHÉ DE 4 VÍAS Y UNA DIRECCIÓN CONCRETA

Computador de 32 bits con **256 MB** de memoria (70 ns), direccionada por bytes. Caché de **64 KB**, líneas de **64 bytes**, **asociativa por conjuntos de 4 vías**. Acceso a caché: 5 ns; penalización por fallo: 100 ns.

a) ¿Cuántos bloques tiene la MP? b) ¿Cuántos conjuntos tiene la caché? c) ¿Qué partes de la dirección son etiqueta, conjunto y byte, y cuántos bits tiene cada una? d) ¿En qué conjunto iría la dirección `0000 0011 1100 0011 0000 0000 1111 1000`? e) ¿Qué tasa de aciertos hace falta para un tiempo medio de 8 ns?

SOLUCIÓN

a) 256 MB / 64 B = 2^{28} / 2^6 = **2^{22} bloques** (4 M bloques).

b) Líneas: 2^{16} / 2^6 = 2^{10} = 1024. Conjuntos: 1024 / 4 = **256**.

c) Byte: 64 = 2^6 → **6 bits**. Conjunto: 256 = 2^8 → **8 bits**. Etiqueta: 32 - 6 - 8 = **18 bits**.



d) Se cortan los bits por la derecha: `000000111100001100` | `00000011` | `111000`. El campo conjunto es 00000011 → **conjunto 3** (y byte 56 dentro de la línea).

e) 8 = 5 + (1 - h) · 100 → 1 - h = 0,03 → **h = 0,97** (97 %).

E2 · TIEMPO MEDIO CON ESCRITURA INMEDIATA

Caché: 4 ns. Memoria principal: 80 ns. Servir un fallo: 120 ns. Escritura inmediata. Tasa de aciertos del 95 % y, de cada 100 accesos, 90 son lecturas. Calcula el tiempo medio de acceso.

SOLUCIÓN

Hay que separar lecturas y escrituras:

- **Lecturas** (90 %): fórmula normal → 4 + 0,05 · 120 = **10 ns**.
- **Escrituras** (10 %): con escritura inmediata **toda escritura llega a la MP**, así que cuesta lo que tarda la MP → **80 ns**.

T_m = 0,9 · 10 + 0,1 · 80 = 9 + 8 = **17 ns**.

Aquí se ve el inconveniente de la escritura inmediata: solo el 10 % de accesos son escrituras y aportan casi la mitad del tiempo.

E3 · TASA DE ACIERTOS PARA 20 NS

Caché de 16 KB con bloques de 32 bytes (8 palabras), tiempo de acceso 10 ns, conectada por un bus de 32 bits a una MP que transfiere un bloque de 8 palabras en 120 ns. Escritura diferida. ¿Qué tasa de aciertos hace falta para un tiempo medio de 20 ns?

SOLUCIÓN

20 = 10 + (1 − h) · 120 → 1 − h = 10 / 120 = 0,0833 → **h ≈ 0,917** (91,7 %).

Con escritura diferida, las escrituras normales se quedan en la caché y no cambian la fórmula. Si te dijeran que el bloque que sale está modificado y hay que volcarlo antes, el fallo costaría 2 × 120 = 240 ns y saldría h ≈ 0,958.

E4 · ESTRUCTURA DE UNA CACHÉ DE 2 VÍAS

Caché de 64 KB, líneas de 64 bytes, asociativa por conjuntos de 2 vías, acceso 20 ns, penalización por fallo 120 ns. Indica: nº de líneas, nº de conjuntos, líneas por conjunto, un dibujo de la estructura y cuánto se tarda en obtener un dato si hay fallo.

SOLUCIÓN

Líneas: 2^{16} / 2^6 = **1024**. Líneas por conjunto = nº de vías = **2**. Conjuntos: 1024 / 2 = **512**.

Conjunto	Vía 0 (etiqueta | 64 bytes)	Vía 1 (etiqueta | 64 bytes)
0	etiq | datos	etiq | datos
1	etiq | datos	etiq | datos
…	…	…
511	etiq | datos	etiq | datos

Con direcciones de 32 bits: byte 6, conjunto 9, etiqueta 17 bits.

Con fallo: hay que traer el bloque (120 ns) y luego servir el dato desde la caché (20 ns) → T_a + T_f = **140 ns**.

E5 · FALLOS DE UN BUCLE EN MIPS

Computador de 32 bits con el juego de instrucciones del MIPS. Caché asociativa por conjuntos de 4 vías, de 32 KB, líneas de 16 bytes, vacía al principio. El código se carga a partir de la dirección `0x00000000`. Calcula el número de fallos y la tasa de aciertos.

```
        li   $t0, 1000          # 0x00 ┐
        li   $t1, 0             # 0x04 │ bloque 0
        li   $t2, 0             # 0x08 │
bucle:  addi $t1, $t1, 1        # 0x0C ┘
        addi $t2, $t2, 4        # 0x10 ┐ bloque 1
        bne  $t1, $t0, bucle    # 0x14 ┘
```

SOLUCIÓN

La diapositiva pone `beq`, pero así el bucle se haría **una sola vez** ($t1 = 1 ≠ 1000 y no salta). Lo que se pretende es repetirlo hasta que $t1 = 1000, que es `bne`.

Accesos: no hay lw/sw, solo instrucciones: 3 + 3 · 1000 = **3003**.

Fallos: 6 instrucciones · 4 bytes = 24 bytes → **2 bloques** de 16 bytes. Cada bloque falla solo la primera vez (32 KB de caché sobran para 2 bloques). → **2 fallos**.

Tasa de aciertos = 3001 / 3003 ≈ **99,93 %**.

Si lo haces literalmente con `beq`: se ejecutan 6 instrucciones, 2 fallos → 4/6 ≈ 66,7 %.

E6 · SUMAR DOS VECTORES DE FLOAT

Direcciones de 32 bits, memoria direccionada por bytes. Caché asociativa por conjuntos de 4 vías, líneas de 64 bytes, tamaño 128 KB. Acceso a caché: 2 ns; tratar un fallo: 80 ns.

```
float v1[10000];
float v2[10000];
for (i = 0; i < 10000; i = i + 1)
    v1[i] = v1[i] + v2[i];
```

a) Tamaño en MB de la memoria direccionable. b) Palabras que caben en la caché. c) Líneas y conjuntos. d) Tasa de aciertos para un tiempo medio de 10 ns. e) Tasa de aciertos del código, contando solo accesos a datos (i está en un registro y la caché empieza vacía).

SOLUCIÓN

a) 2^{32} bytes = $2^{12} \cdot 2^{20}$ B = **4096 MB** (4 GB).

b) 128 KB / 4 B = $2^{17} / 2^{2} = 2^{15}$ = **32 768 palabras**.

c) Líneas: $2^{17} / 2^{6}$ = **2048**. Conjuntos: 2048 / 4 = **512**.

d) $10 = 2 + (1 - h) \cdot 80 \rightarrow 1 - h = 0{,}1 \rightarrow$ **h = 0,9** (90 %).

e) Cada vuelta hace **3 accesos** a datos: leer v1[i], leer v2[i] y escribir v1[i] → 3 · 10 000 = **30 000 accesos**.
Un float ocupa 4 bytes → en una línea de 64 bytes caben **16 floats**. Cada vector ocupa 40 000 B / 64 = **625 líneas**.
Se recorren en orden: al empezar cada bloque de v1 falla la lectura de v1[i] (y luego la escritura ya acierta), y al empezar cada bloque de v2 falla su lectura. Fallos = 625 + 625 = **1250**. Al ser de 4 vías, los bloques de v1 y v2 no se echan entre sí.
Tasa de aciertos = (30 000 - 1250) / 30 000 = 28 750 / 30 000 ≈ **95,8 %**.

Comprobación por bloques: cada 16 vueltas hay 48 accesos y 2 fallos → 46/48 = 95,8 % ✓

¡OJO! FALLOS TÍPICOS EN ESTOS EJERCICIOS

- Olvidar que **KB, MB… son potencias de 2** en memoria: 64 KB = 2^{16}, no 64 000.
- Usar el número de **líneas** en vez del de **conjuntos** para el campo central en las cachés por conjuntos.
- Poner h en porcentaje dentro de la fórmula: va en **tanto por uno** (0,95, no 95).
- Al contar accesos, olvidar el acceso extra de cada `lw` / `sw`, o la última comparación que sale del bucle.

18 Chuleta y erratas del Tema 4

18.1 Chuleta

Qué	Cómo
Accesos a memoria de un programa	1 por instrucción ejecutada + 1 por cada lw/sw ejecutado
Tasa de aciertos h	aciertos / accesos totales
Tiempo medio de acceso	$\mathbf{T_m = h \cdot T_a + (1-h) \cdot (T_a + T_f) = T_a + (1-h) \cdot T_f}$
Despejar h	$1 - h = (T_m - T_a) / T_f$
Nº de bloques de MP	tamaño MP / tamaño bloque
Nº de líneas	tamaño caché / tamaño línea
Nº de conjuntos	nº líneas / nº vías
Bits de byte	$\log_2$(tamaño de línea)
Bits de línea (directa)	$\log_2$(nº líneas)
Bits de conjunto	$\log_2$(nº conjuntos)
Bits de etiqueta	bits de dirección - los demás campos
Línea / conjunto de un bloque B	B mod nº líneas / B mod nº conjuntos

Tipo	Campos
Directa	etiqueta | línea | byte
Asociativa	etiqueta | byte
Por conjuntos	etiqueta | conjunto | byte

Memoria	Tecnología	Se usa en
SRAM	interruptor, sin refresco	caché
DRAM	condensador, con refresco	memoria principal
ROM	no volátil	BIOS

Potencia	2^{10}	2^{16}	2^{20}	2^{30}	2^{32}
Valor	1 K	64 K	1 M	1 G	4 G

18.2 Erratas corregidas del Tema 4

Esto es lo que estaba mal (o liaba) en las diapositivas. En la guía ya está corregido en su sitio.

Dónde	Pone	Correcto
T4 (I) · diap. 14 y 17	El bucle usa `bgt` (y en la 14, `fin: li $t1, 1`)	`bge`, como en la diapositiva anterior. Con `bgt` el bucle daría 1001 vueltas (y escribiría fuera del vector) y la cuenta no saldría
T4 (I) · diap. 26	"25.6 billones" y "12.8 billones" de accesos	25,6 y 12,8 **mil millones** (es el *billion* inglés, 10^9; un billón español es 10^{12})

T4 (II) · diap. 4	`mult $t2 $t0 4`	`mul $t2, $t0, 4`: `mult` solo admite dos registros y deja el resultado en HI/LO
T4 (II) · diap. 18	480480 ns = 480.480 ms	480 480 ns = 480,48 µs ≈ **0,48 ms**
T4 (II) · diap. 43	`addi $t3, $t3, $t4`	`add $t3, $t3, $t4` (`addi` necesita un número, no un registro)
T4 (II) · diap. 54	"5 = 4 × h + (120 + 4) × (1 - h) = 16 ns ⇒ h > 0.9916" y no resuelve el caso de 10 ns	5 = 4h + 124(1 - h) ⇒ h ≈ 0,9917 (el "= 16 ns" sobra). Para 10 ns: h > 0,95
T4 (II) · diap. 62	2^{26} B = 64 megabloques	2^{26} **bloques** (no bytes)
T4 (II) · diap. 63-64	Palabra de dirección `0x00000064`, pero el binario y la flecha señalan la dirección 64	64 en decimal es `0x00000040` (`0x64` = 100)
T4 (II) · diap. 99 y 104	Caché de 64 KB con líneas de 8 B: "2^{14} líneas"; campos directa 15 | 14 | 3	$2^{16}/2^3$ = **2^{13} líneas**; campos **16 | 13 | 3** (los números de la diapositiva son los de una caché de 128 KB)
T4 (II) · diap. 117	64 KB, 4 vías: "2^{12} conjuntos"; campos 17 | 12 | 3	$2^{13}/4$ = **2^{11} conjuntos**; campos **18 | 11 | 3**
T4 (II) · diap. 132	`beq $t1, $t0, bucle`	`bne`: con `beq` el bucle solo se ejecuta una vez
Aclaración	Tiempo de acceso a MP: 40-100 ns (T4-I), 70-100 ns (diap. 22) y 20-50 ns (T4-II diap. 45)	No es un error: son fuentes y épocas distintas. Quédate con "decenas de ns" para la DRAM y "~1 ns" para la SRAM
Aclaración	"Post-escritura" (diap. 120) y "escritura diferida" (diap. 121)	Son **lo mismo** (*write-back*). La otra es la escritura inmediata (*write-through*)
Varios	"computdor", "Tipos e memoria", "Memoria se solo lectura", "aleaotorio", "Funcionamieto", "funcioamiento", "AICERTO", "Ejerccio", "correspondicena", "memorial principal", "Fundamenros"	computador, tipos de memoria, de solo lectura, aleatorio, funcionamiento, acierto, ejercicio, correspondencia, memoria principal, fundamentos

Si en clase os lo explicaron de otra manera, haz caso al profe. ¡Mucho ánimo! ♥